

Digital Design with Verilog

FSM Design Examples with Verilog

A. Sahu

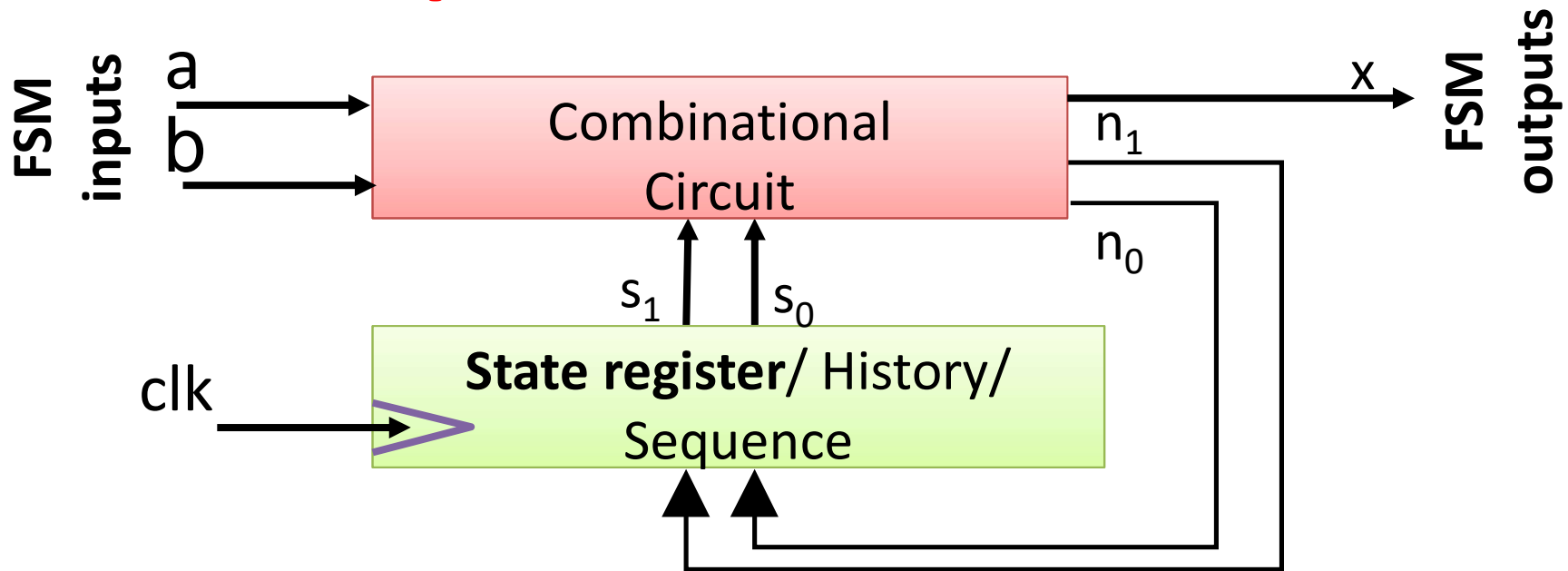
Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

Outline

- FSM Examples
- Verilog Model

Sequential Circuit: FSM



Formally Describe/mathematically Describe

Boolean Algebra: Working of Combinational Circuit
Finite State Machine: Working of Sequential Circuit

$X(t) = F(a(t), b(t), H)$
H is History/Sequence/State

Set Theoretic Description

Moore Machine is an ordered quintuple

$$M o o r e = (S, I, O, \delta, \lambda)$$

where

$$S = \{s_1, s_2, \dots, s_n\}$$

$$I = \{i_1, i_2, \dots, i_m\}$$

$$O = \{o_1, o_2, \dots, o_l\}$$

$$\delta = S \times I \rightarrow S$$

$$\lambda = S \rightarrow O$$

S= finite set of state ($\neq \phi$)

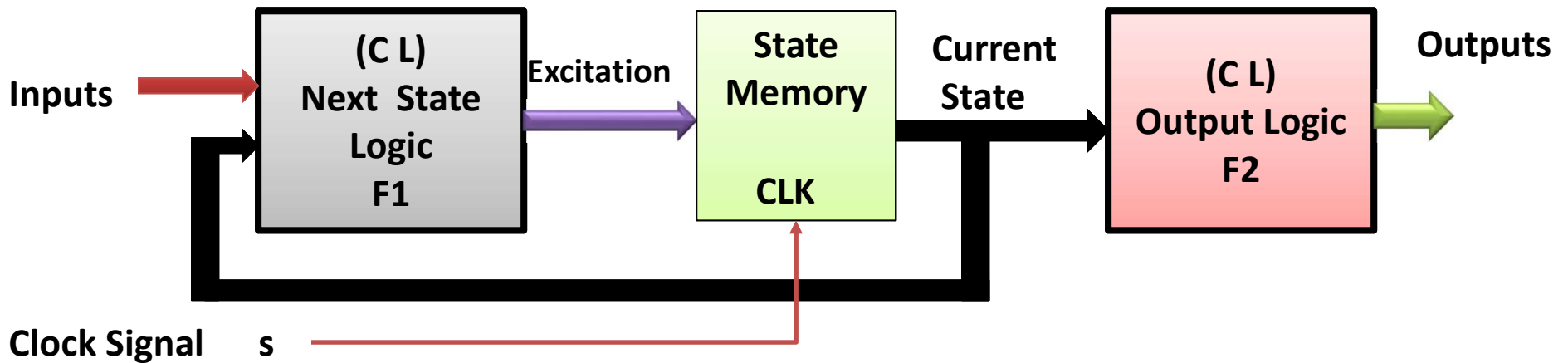
I = finite set of inputs ($\neq \phi$)

O=finite set of output ($\neq \phi$)

δ =next state function

λ = output function

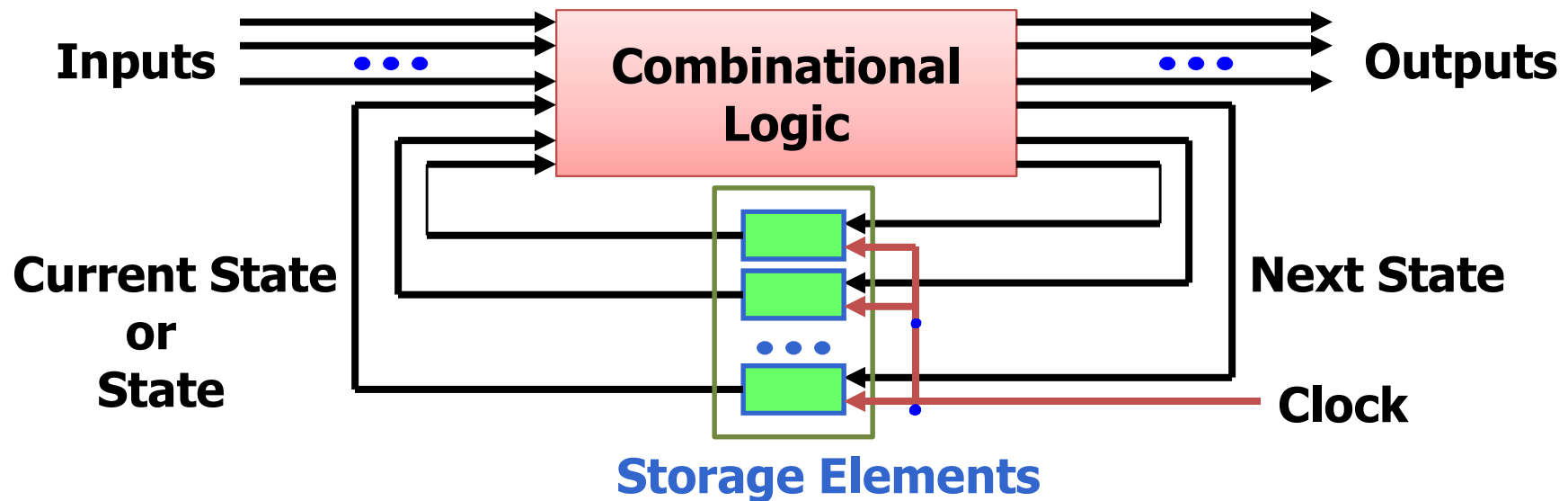
Moore Machine: Moore FSM



Next state = $F_1(\text{current state, inputs})$
Output = $F_2(\text{current state})$

Clocked synchronous FSM structure

- **States:** Determined by possible values in sequential storage elements
- **Transitions:** Change of state
- **Clock:** Controls when state can change by controlling storage



Standard Procedure for Sequential Circuit Design

Sequential Circuit Design Process

- Step 1: Describe problem in HLL English like statements
- Step 2: Model the problem using FSM
 - **Step 2.A: (optional) Minimize FSM**
- Step 3: Represent the FSM in Tabular Form
- Step 4: Derive number of FF require

Sequential Circuit Design Process

- Step 5: State Encoding
 - **Step 6.A: Optional Encoding Optimization**
- Step 6: Derivation of
 - Next State : from Present State and Input
 - Output : from Present State (for Mealy FSM: including input)
- Step 7: Derive combination function for NS logic, OP logic
- Step 8: Realize FSM
 - by Connecting the NS, OP logic, State Reg and Clock

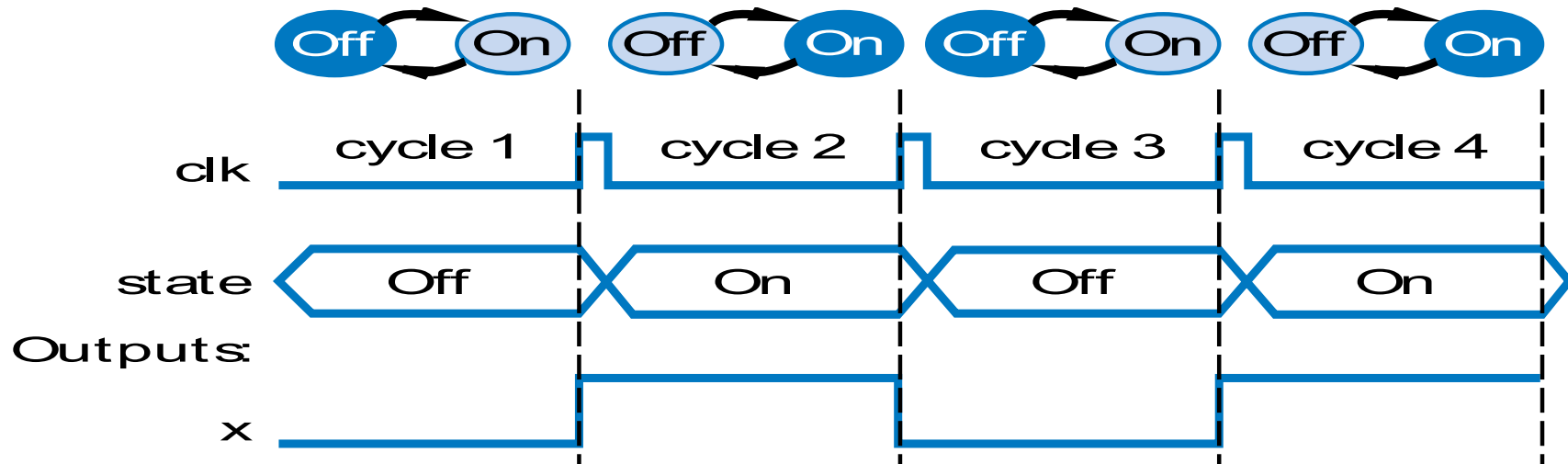
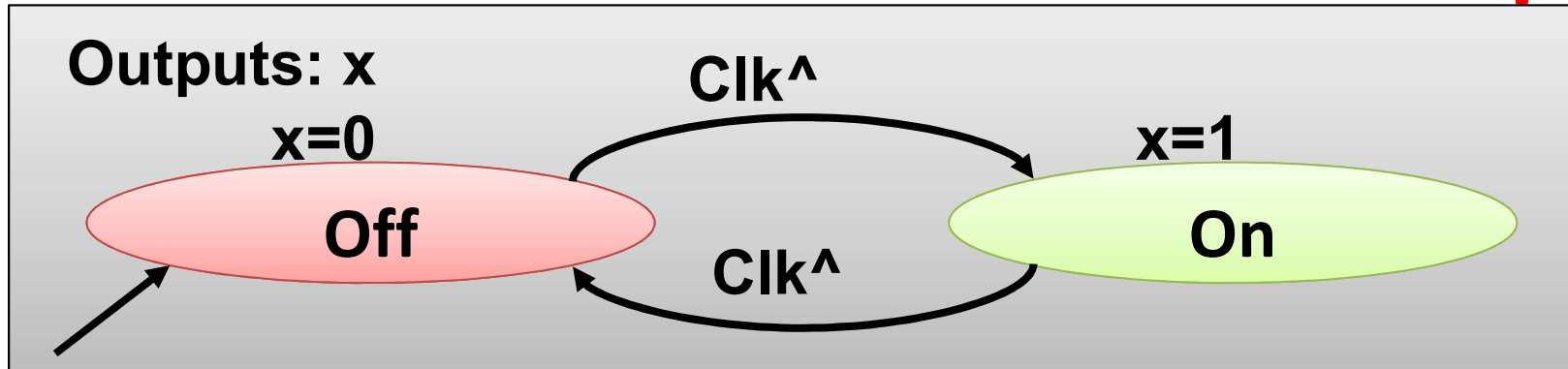
FSM Example 1

ON-OFF FSM

Finite State Machine: Example

- Step1: English like statement
 - Make x change toggle (0 to 1, or 1 to 0) every clock cycle
- Step2: Model the problem using FSM
 - Two states: “Off” ($x=0$), and “On” ($x=1$)
 - Transition on rising clock edge
 - from Off to On, or On to Off
 - Arrow with no starting state points to
 - initial state (when circuit first starts)

Moore Finite State Machine: Example



FSM

We often draw FSM graphically, known as *state diagram*

Can also use table (state table), or textual languages

Controller for On-Off

Step 3: Represent the FSM in Tabular Form

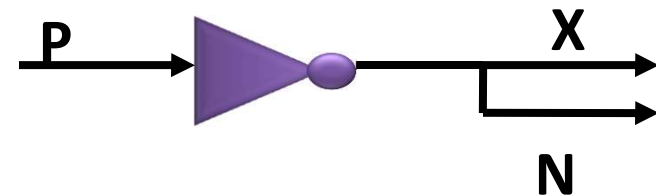
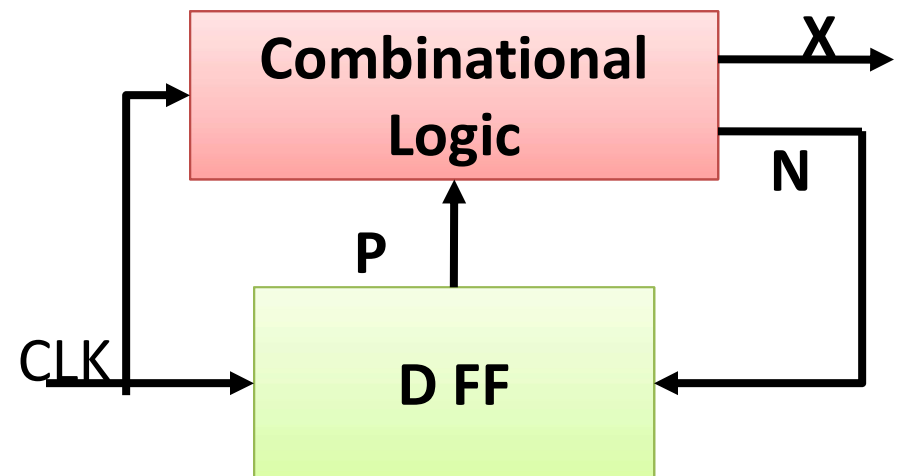
Input		Output	
CLK	PS	X	NS
RE 1	0	1	1
RE 1	1	0	0

Step 4: Derive number of FF require

Two States: 1 FF

Step 5: State Encoding

State 0: 0 State 1: 1



Rising Edge: Clock implicit

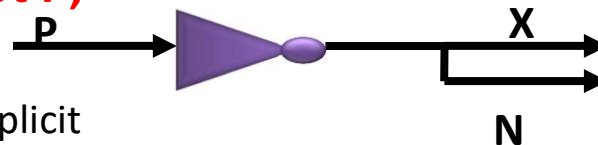
Controller for On-Off

Input		Output	
CLK	P	X	N
RE 1	0	1	1
RE 1	1	0	0

Step 6: Derivation of
 Next State : from Present State and Input
 Output : from Present State

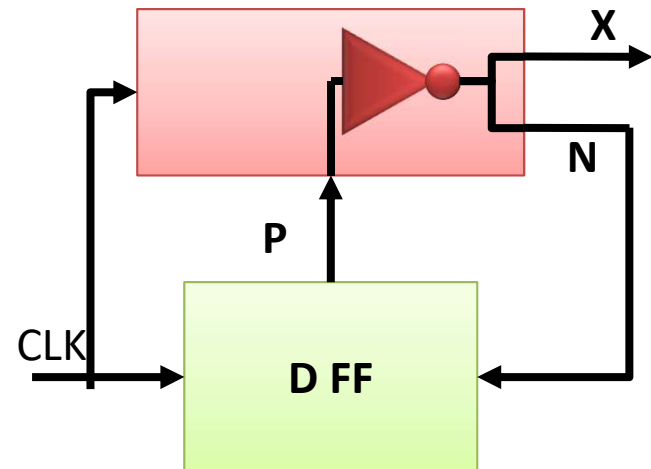
Step 7: Derive combination function for
 NS logic, OP logic

NS = not P; X=not P;



Rising Edge: Clock implicit

Step 8: Realize FSM
 by Connecting the NS, OP logic, State Reg and
 Clock



FSM Example 2

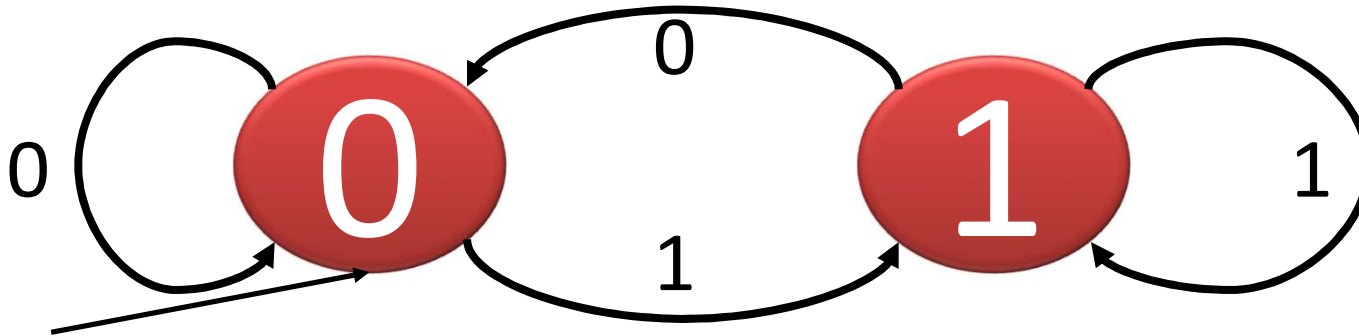
D-FF

Finite State Machine: D FF

- Step1: English like statement
 - When NS is depends on value of Data input
- Step2: Model the problem using FSM
 - Two states: “Off” ($S=0$), and “On” ($S=1$)
 - Transition on rising clock edge
 - Arrow with no starting state points to
 - initial state (when circuit first starts)

FSM for D-FF

Step2: Model the problem using FSM



Step 3: Represent the FSM in Tabular Form

$Q(t+1)=Q(t)$

State: 0, 1

Input : 0, 1

Output: 0,1

PS= Q(t)	Input (D)	NS =Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

FSM Controller for D-FF is D-FF

Step 4: Derive number of FF require

Two States: 1 FF : Obvious

Step 5: State Encoding

State 0: 0 State 1: 1

Step 6: Derivation of Next State

PS= Q(t)	Input	NS =Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

Step 7: Derive for NS logic, OP logic

NS = D; X=D;

$$Q(t+1)=Q(t)$$

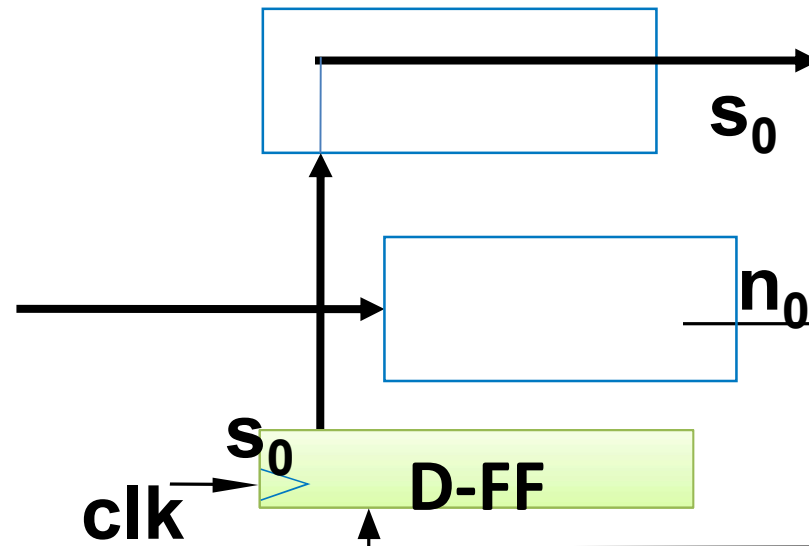
State: 0, 1

Input : 0, 1

Output: 0,1

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



FSM Example 3

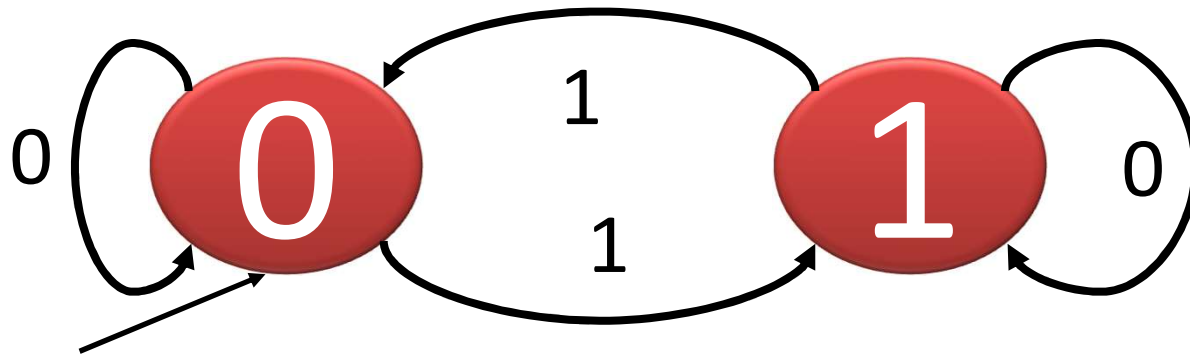
T-FF

Finite State Machine: T FF

- Step1: English like statement
 - When NS toggle every clock when input T is one, else stay with same present state
- Step2: Model the problem using FSM
 - Two states: “Off” ($S=0$), and “On” ($S=1$)
 - Transition on rising clock edge
 - Arrow with no starting state points to
 - initial state (when circuit first starts)

FSM for T-FF

Step2: Model the problem using FSM



State: 0, 1

Input : 0, 1

Output: 0,1

Step 3: Represent the FSM in Tabular Form

PS= Q(t)	Input	NS =Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Step 4: Derive number of FF require

Two States: 1 FF : Obvious

Step 5: State Encoding

State 0: 0 State 1: 1

FSM Controller for T-FF

Step 6: Derivation of Next State

PS= Q(t)	Input	NS =Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Step 7: Derive for NS logic, OP logic

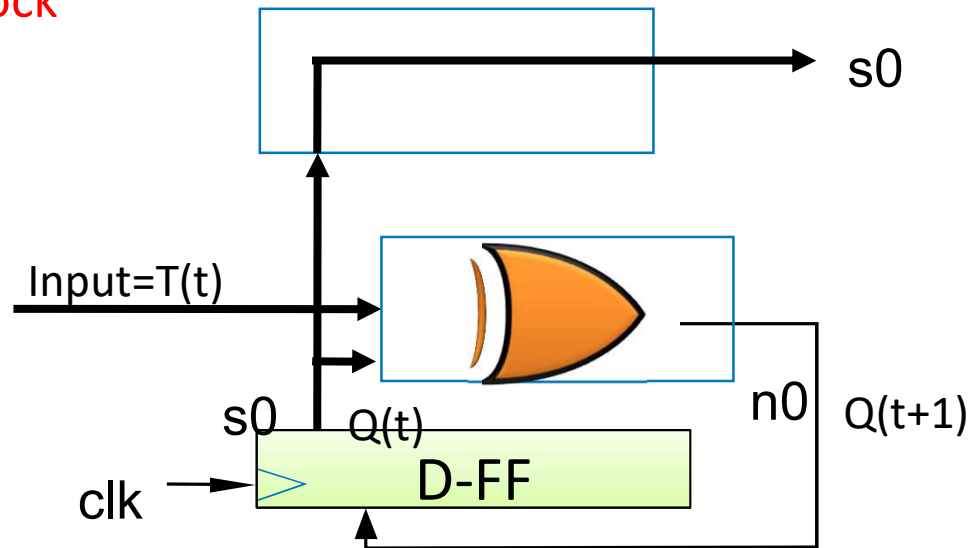
$$NS = Q(t)'T(t) + Q(t)T'(t) \\ = Q(t) \text{ xor } T(t)$$

$$X=Q;$$

$$Q^+ = QT' + Q'T$$

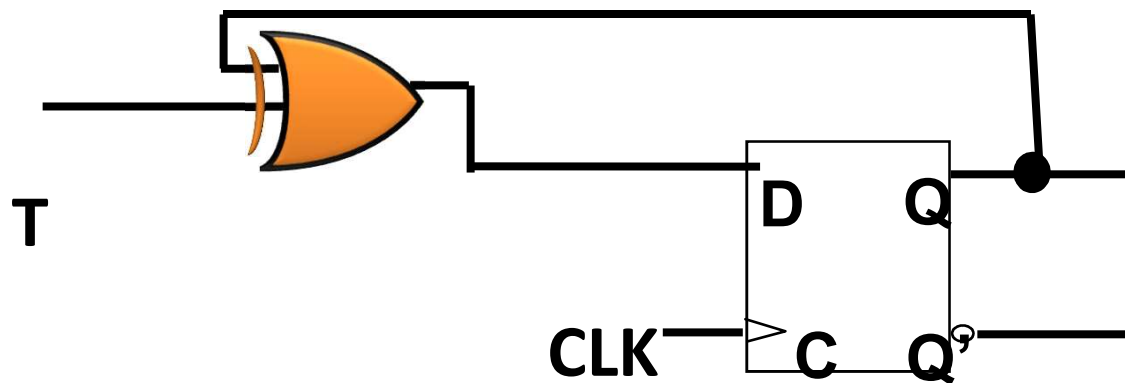
Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



FSM Controller for T-FF

Same as : Given a D FF, Construct T FF

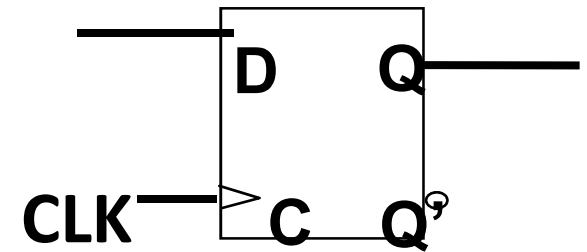


T	D	$TQ' + T'Q$
0	Q	$0Q' + 1Q$
1	Q'	$1Q' + 0Q$

$$D = T Q' + T' Q$$

Verilog Code for FSM of T-FF

```
//Code for Data Flipflop; will be used in FSM  
implementation  
module DFF( input wire D, input wire clk, input rst,  
             output reg Q)  
  always @ (posedge rst) begin  
    if (rst) Q=0;  
  end  
  always @(posedge clk) begin  
    if (!rst) Q <= D;  
  end  
endmodule
```



Verilog Code for FSM of T-FF

```
module tff_fsm (input clk, input t, input rst, output q);  
    wire NS, S;  
  
    DFF D0(NS, clk, rst, S);  
    assign NS = t ^ S; //NS logic  
    assign q = S; //output logic  
  
endmodule
```

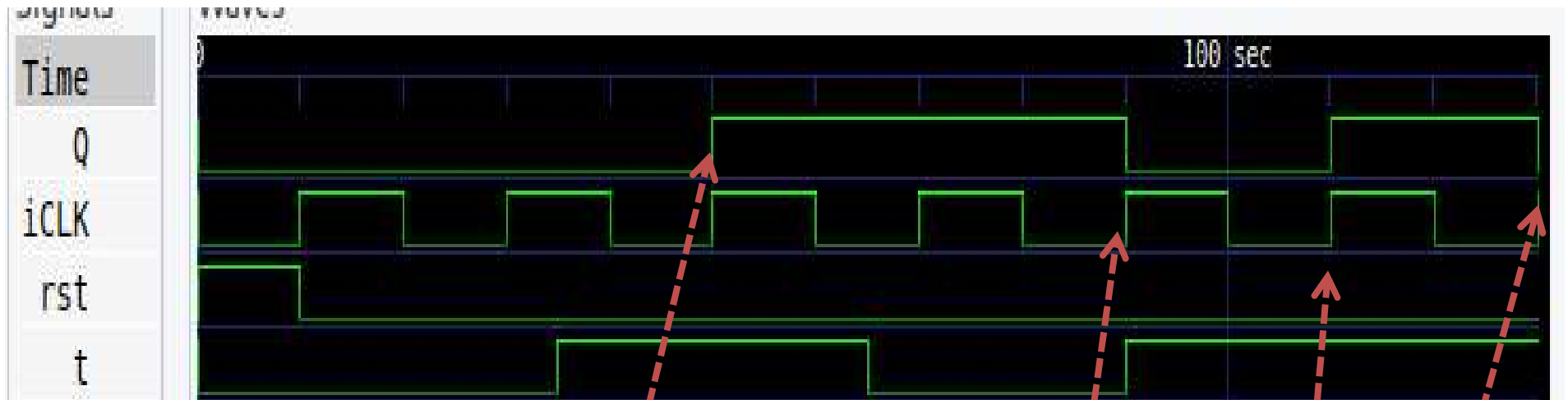
Verilog Code for FSM of TFF: Test bench

```
module tff_fsm_test;
reg t, iCLK, rst;
wire Q;
integer i;
tff_fsm T0(iCLK, t, rst, Q); // Inst
//Part I
//Part II
initial
    $monitor ("iCLK = %b, t = %b, Q = %b",
              iCLK, t, Q);
end
endmodule
```

```
initial begin
    iCLK=0;
    for(i=0;i<20; i++) begin
        #10; iCLK = ~iCLK;
    end
end
```

```
initial begin
    $dumpfile("tff_fsm.vcd");
    $dumpvars;
    t = 0;rst=1;
    #10 rst=0;
    #25 t = 1;
    #30 t = 0;
    #25 t = 1;
    #40 $finish;
end
```

Waveform T-FF



Q changes When T = 1

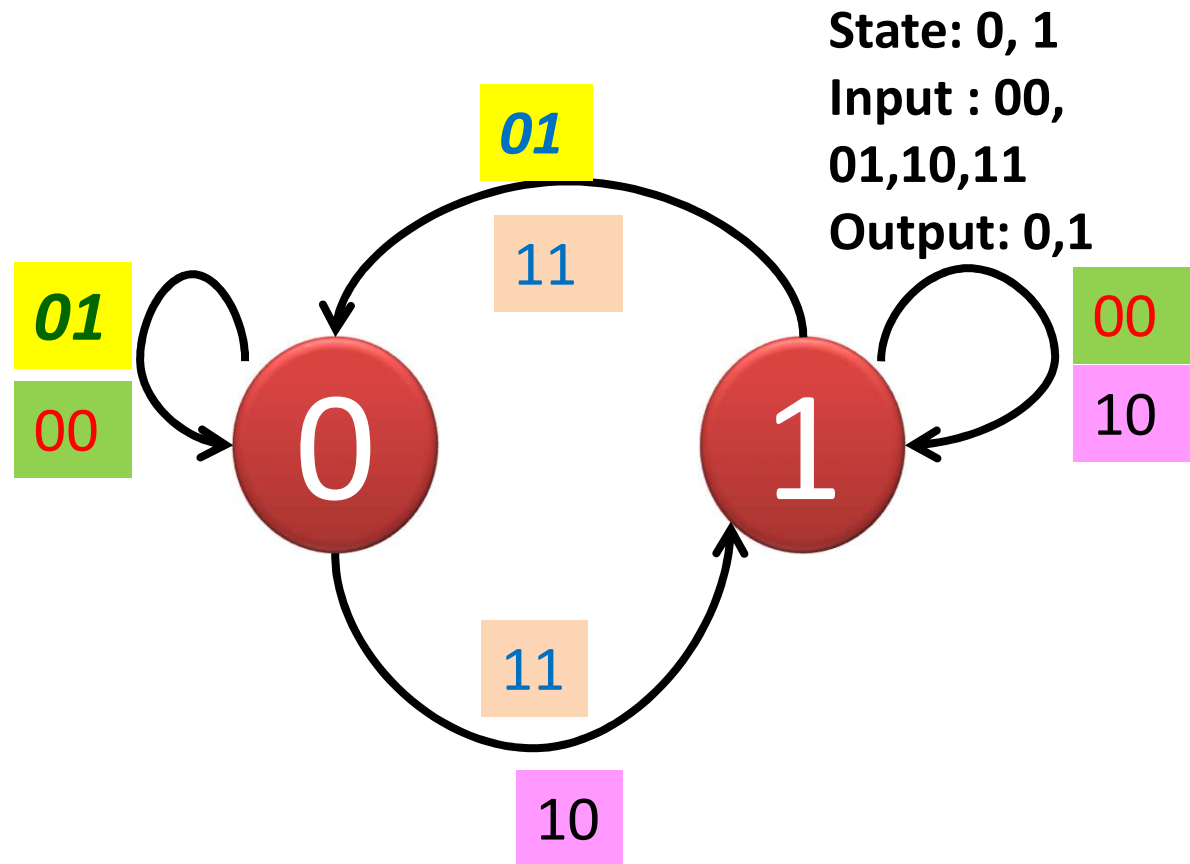
FSM Example 3

JK-FF

Finite State Machine: JK FF

- Step1: English like statement
 - When output reset when $K=1, J=0$, toggle when $JK=11$, set to 1 when $J=1, k=0$ and no change for $JK=00$
- Step2: Model the problem using FSM
 - Two states: “Off” ($S=0$), and “On” ($S=1$)
 - Transition on rising clock edge
 - Arrow with no starting state points to
 - initial state (when circuit first starts)

FSM for JK-FF



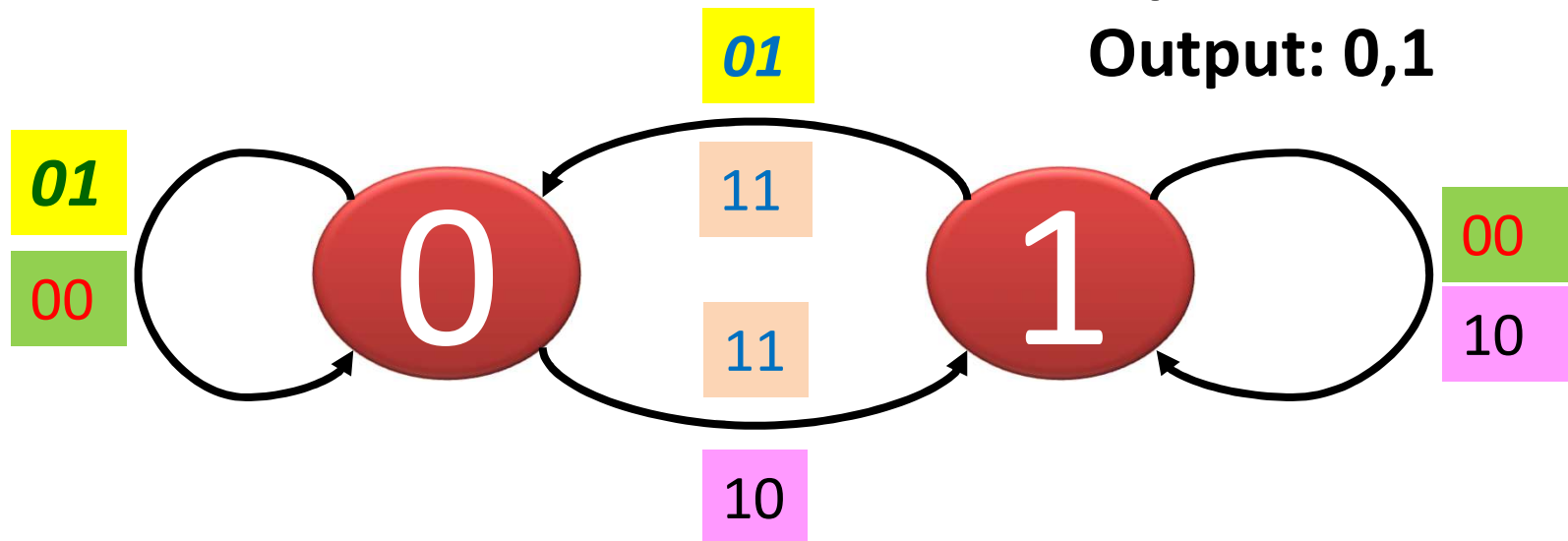
FSM for JK-FF

Step2: Model the problem using FSM

State: 0, 1

Input : 00, 01,10,11

Output: 0,1



Step3,4,5: Trivial

FSM for JK-FF

Step 6: Derivation of Next State

PS= Q(t)	Input(JK)	NS =Q(t+1)
0	00	0
0	01	0
0	10	1
0	11	1
1	00	1
1	01	0
1	10	1
1	11	0

State: 0, 1

Input : 00, 01,10,11

Output: 0,1

Step 7: Derive for NS logic, OP logic

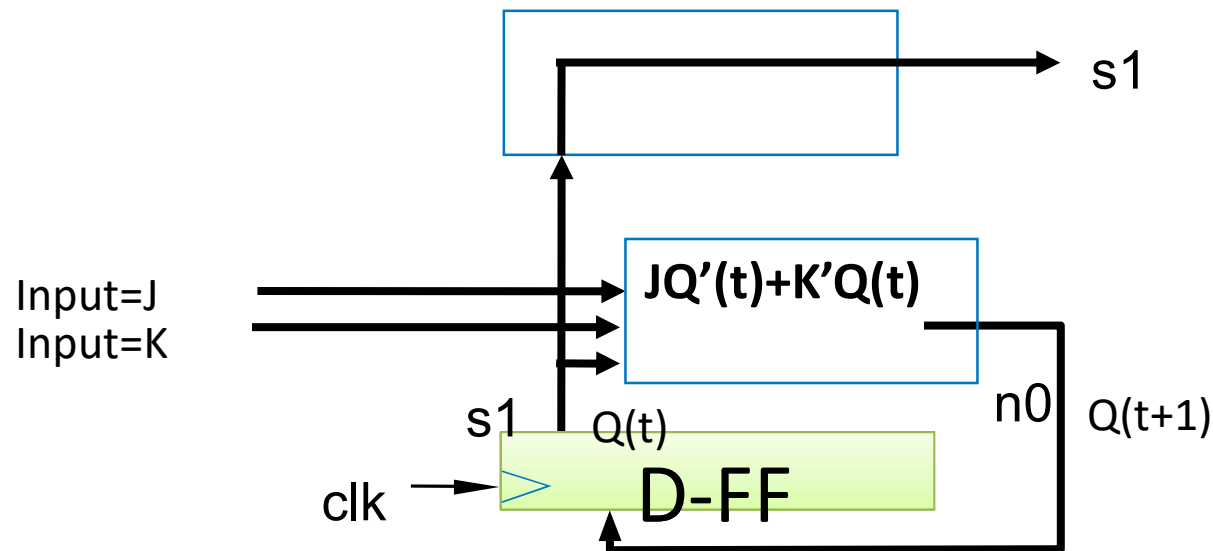
$$NS = JQ'(t) + K'Q(t); X=Q;$$

FSM Controller for JK-FF

Step 8: Realize FSM

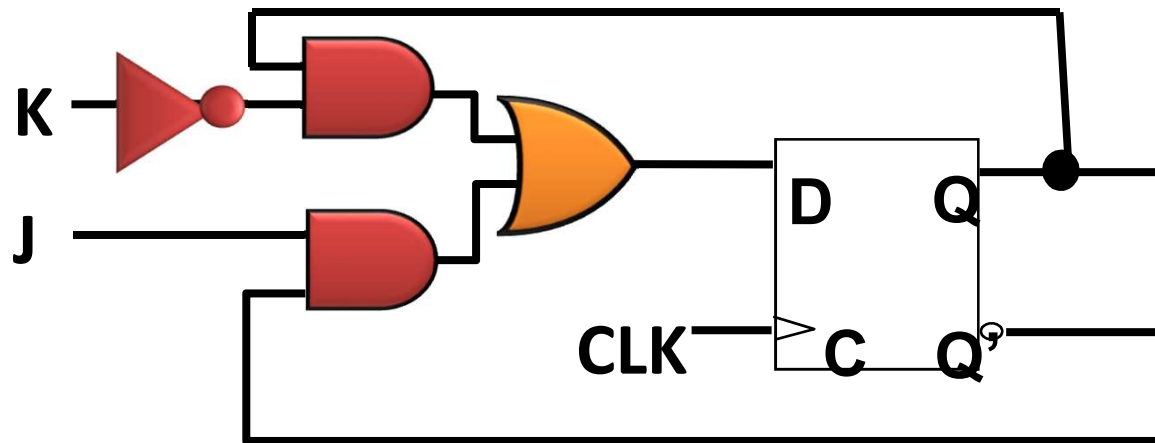
by Connecting the NS, OP logic, State Reg and Clock

$$Q(t+1) = JQ'(t) + K'Q(t)$$



FSM Controller for JK-FF

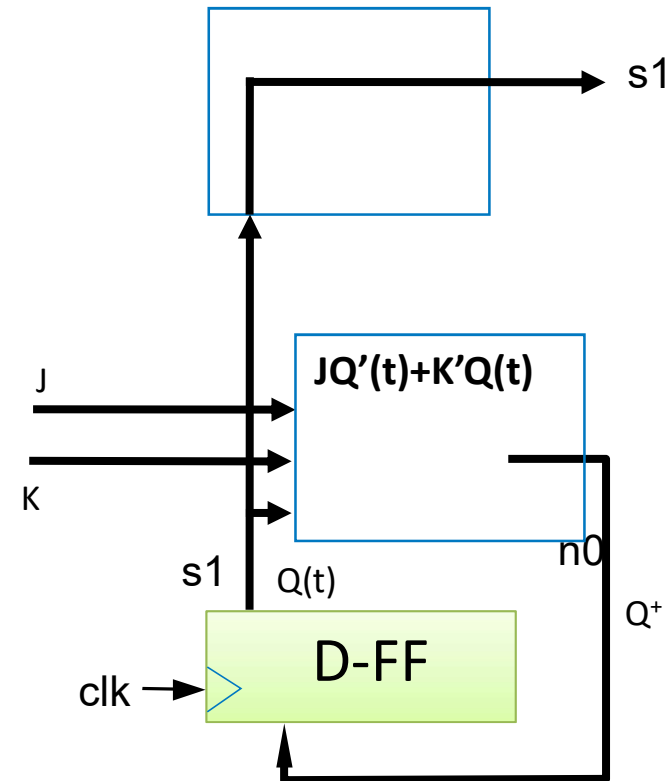
Same as : Given a D FF, Construct JK FF



$$D = JQ'(t) + K'Q(t)$$

Verilog Code for FSM of JK-FF

```
module jkff_fsm (input clk, input j,  
                input k, input rst, output q);  
    wire NS, S;  
  
    DFF D0(NS, clk, rst, S);  
    assign NS = (j&~q) | (~k&q); //NS logic  
    assign q = S; //output logic  
  
endmodule
```



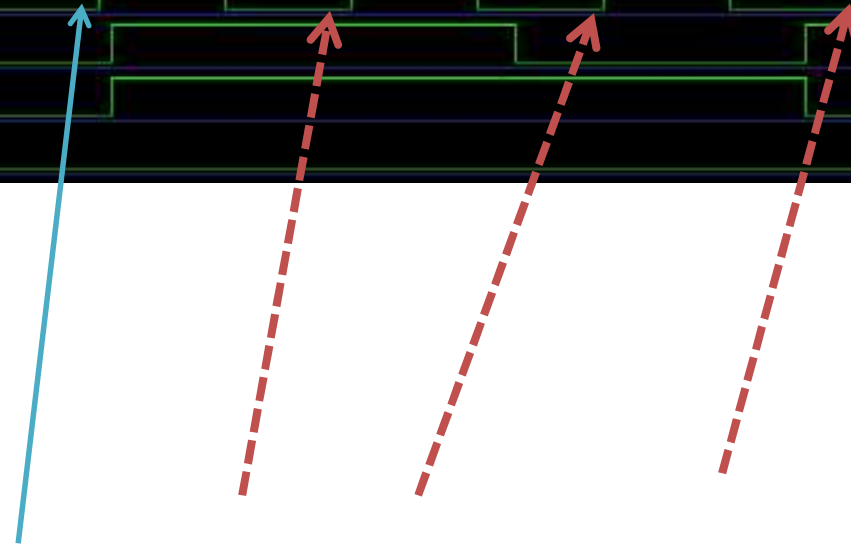
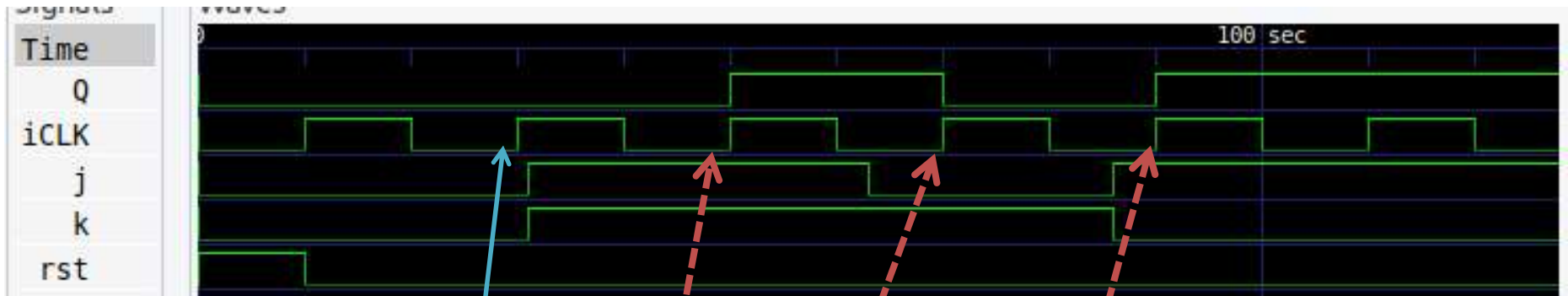
Verilog Code for FSM of JKFF: Test bench

```
module tff_fsm_test;
reg j, k, iCLK, rst;
wire Q;
integer i;
jkff_fsm JK0(iCLK, j, k, rst, Q); // Inst
//Part I
//Part II
initial
    $monitor ("iCLK = %b, j = %b, k=%b, Q = %b",
        iCLK, j, k, Q);
end
endmodule
```

```
initial begin
    iCLK=0;
    for(i=0;i<20; i++) begin
        #10; iCLK = ~iCLK;
    end
end
```

```
initial begin
    $dumpfile("jkff_fsm.vcd");
    $dumpvars;
    j = 0; k=0; rst=1;
    #10 rst=0;
    #21 j = 1; k =1;
    #32 j = 0; k =1;
    #23 j = 1; k =0;
    #42 $finish;
end
```

Waveform T-FF



Q changes When rising edge

FSM Example 4

RS-FF

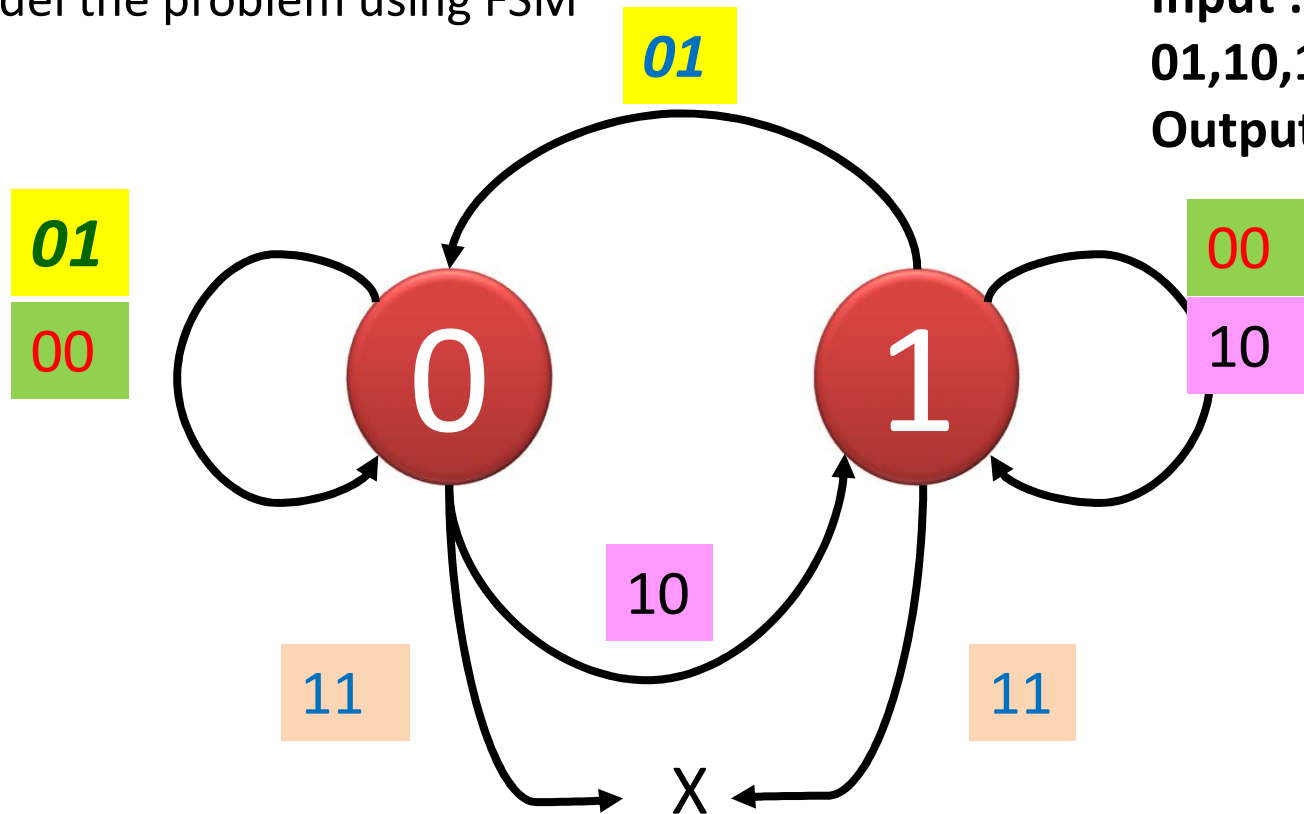
FSM for RS-FF

Step2: Model the problem using FSM

State: 0, 1

Input : 00,
01,10,11

Output: 0,1



FSM for RS-FF

Step 6: Derivation of Next State

PS= Q(t)	Input(RS)	NS =Q(t+1)
0	00	0
0	01	0
0	10	1
0	11	x
1	00	1
1	01	0
1	10	1
1	11	x

State: 0, 1

Input : 00, 01,10,11

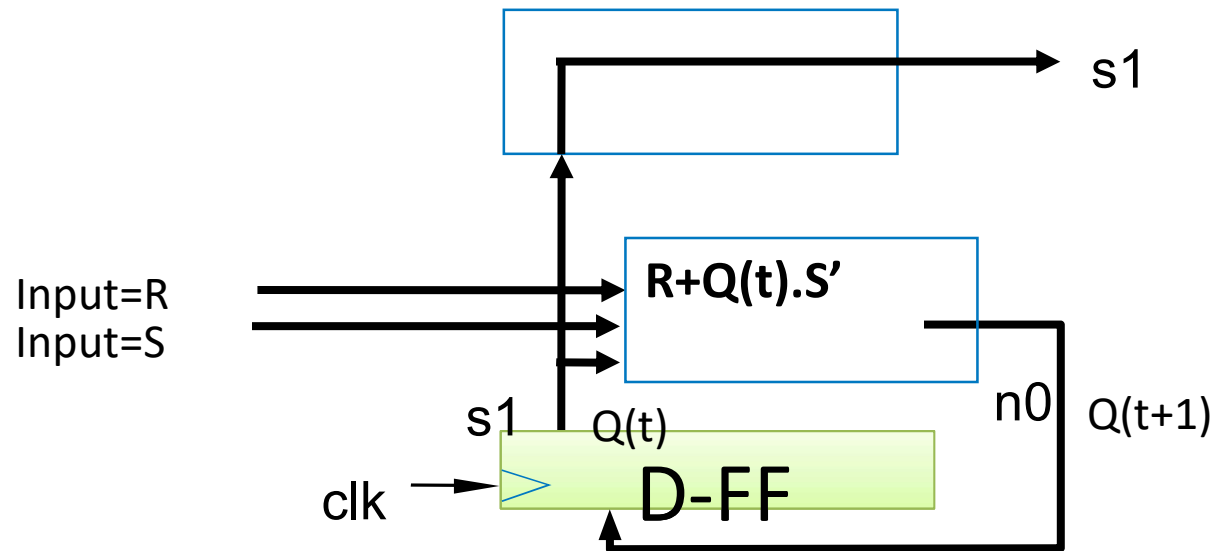
Output: 0,1

$$Q(t+1)=R+Q(t).S'$$

FSM Controller for RS-FF

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



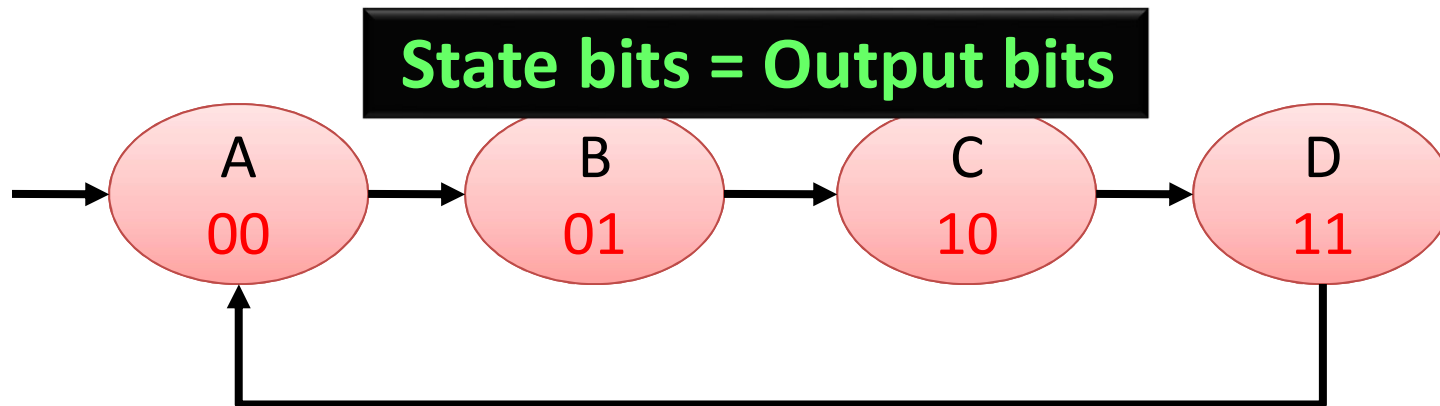
Same as
Given a D FF, Construct
RS FF
 $Q(t+1) = R + Q(t) \cdot S'$

FSM Example 5

(General : Two bit binary Counter)

Two bit counter: 00,01,10,11,repeat

- Step1: Want 00, 10, 10, 11, repeat
 - Each value for one clock cycle
- Step2: Can describe as FSM: Four states,
Transition on rising clock edge to next state



FSM of Counter : 2 bit

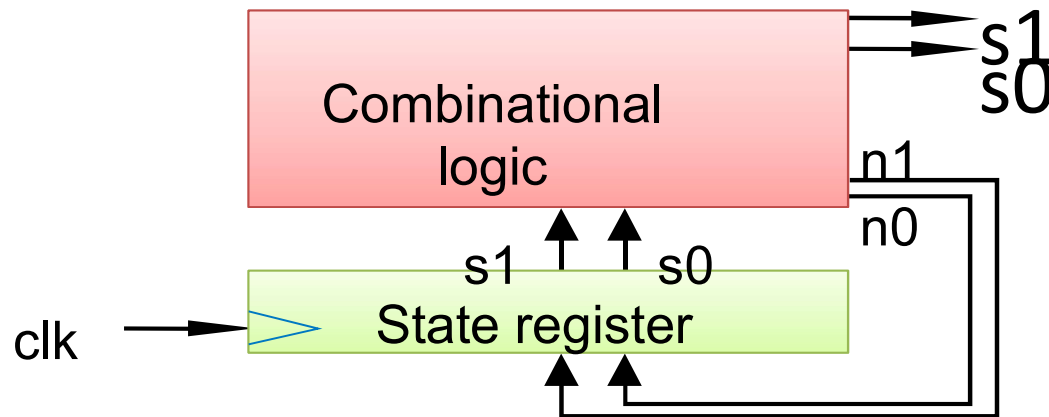
Step 3: Represent the FSM in Tabular Form

Step 4: Derive number of FF require

Four States: 2 FF : Obvious

Step 5: State Encoding

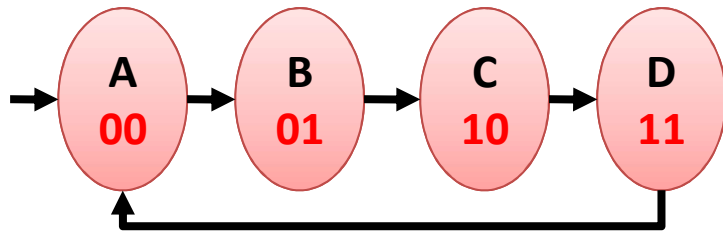
State A: 00 State B: 01,
State C: 10, State D:11



	I/P		O/P	
	s1	s0	n0	n1
A	0	0	0	1
B	0	1	1	0
C	1	0	1	1
B	1	1	0	0

Step 6: Derivation of Next State

FSM Controller: Binary Counter



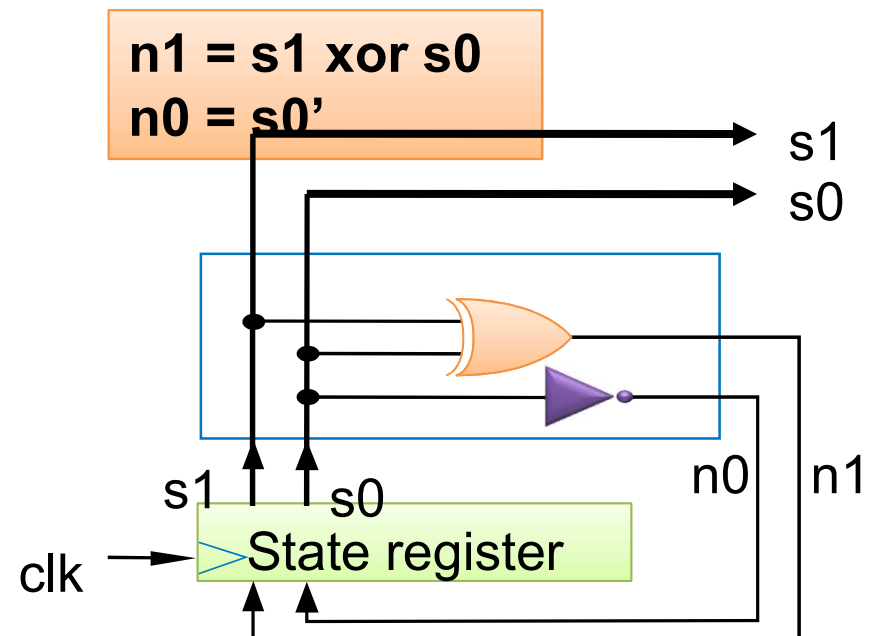
Step 7: Derive for NS logic, OP logic

NS/OP : $n1 = s1 \text{ xor } s0$; $n0 = s0'$

	I/P		O/P	
	s1	s0	n0	n1
A	0	0	0	1
B	0	1	1	0
C	1	0	1	1
D	1	1	0	0

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



Verilog Code for FSM of 2 bit CTR

```
module ctr2bit_fsm (input clk, input rst, output q1, output q0);  
    wire NS0, NS1, NS2, S0, S1, S2;  
  
    DFF D0(NS0, clk, rst, S0); //Instantiation  
    DFF D1(NS1, clk, rst, S1); //Instantiation  
  
    assign NS1 = S1 ^ S0; //NS logic  
    assign NS0 = ~S0;      //NS logic  
    assign q1 = S1;        //output logic  
    assign q0 = S0;        //output logic  
endmodule
```

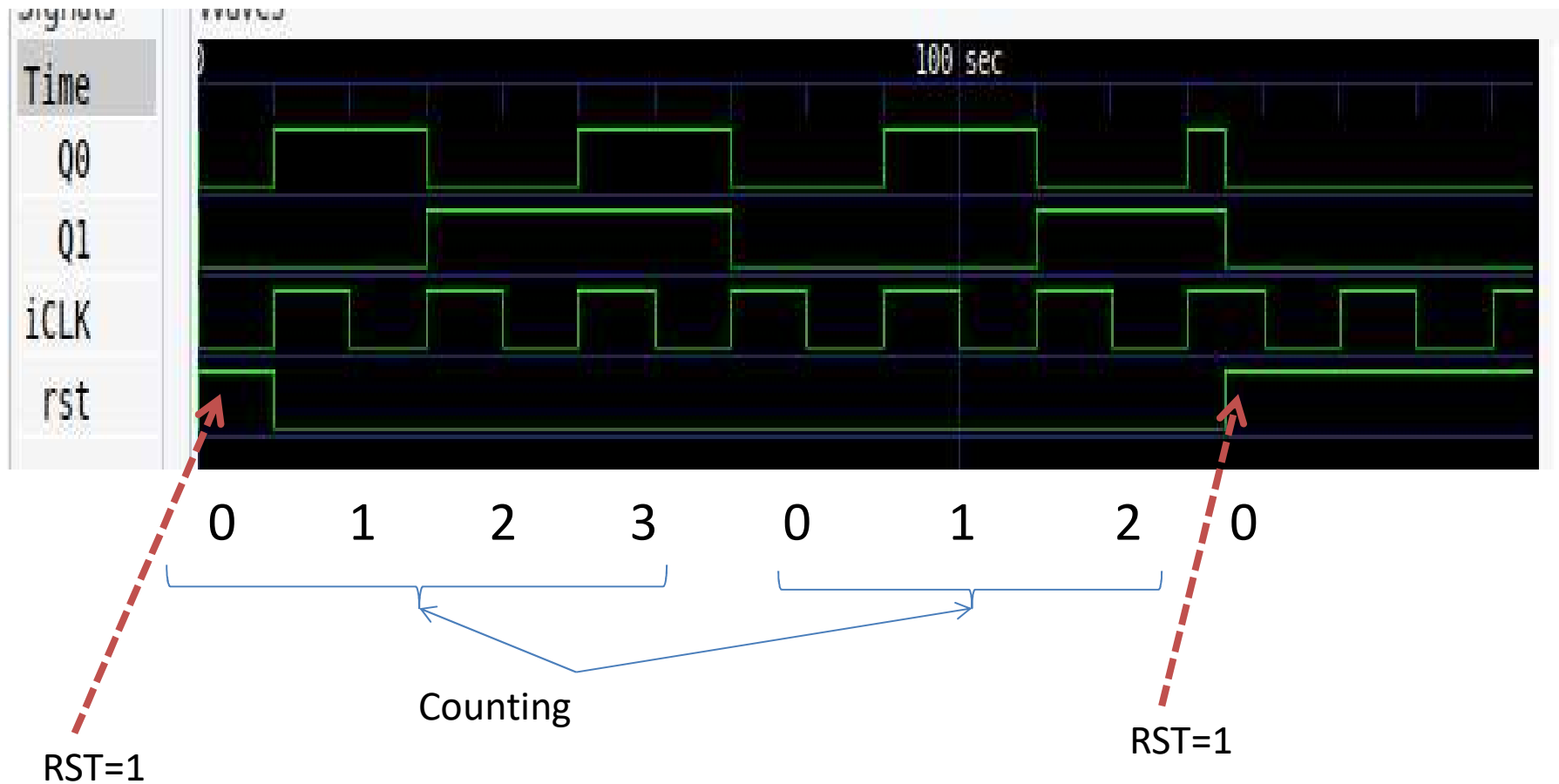
Verilog Code for FSM of Ctr: Test bench

```
module 2bitctr_fsm_test;
reg iCLK, rst;
wire Q1, Q0;
integer i;
ctr2bit_fsm CTR2(iCLK, rst, Q1, Q0); //
//Part I
//Part II
initial
    $monitor ("iCLK = %b, Q1 = %b, Q0 = %b",
              iCLK, Q1, Q0);
end
endmodule
```

```
initial begin
    iCLK=0;
    for(i=0;i<20; i++) begin
        #10; iCLK = ~iCLK;
    end
end
```

```
initial begin
    $dumpfile("ctr2bit_fsm.vcd");
    $dumpvars;
    rst=1;
    #10 rst=0;
    #125 rst = 1;
    #40 $finish;
end
```


Waveform 2 bit counter

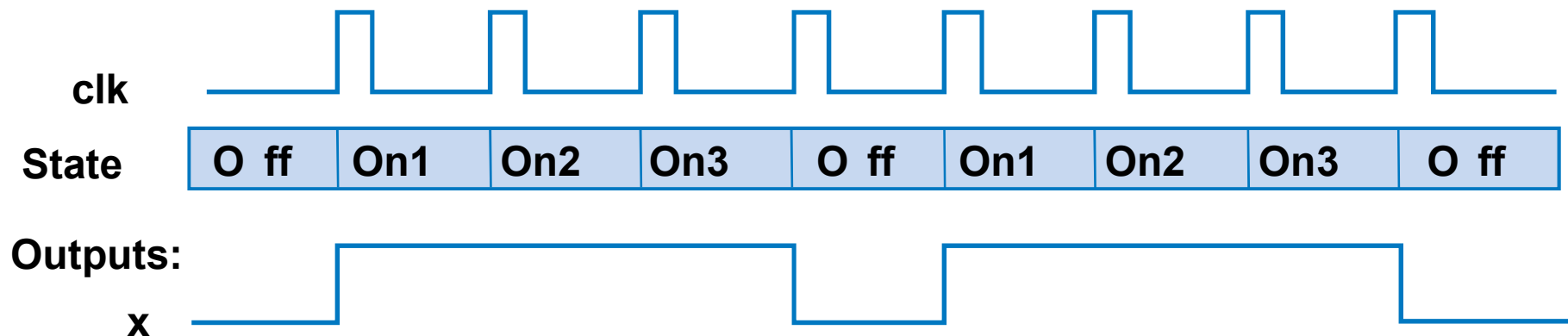


FSM Example 6

(General : sequence generator)

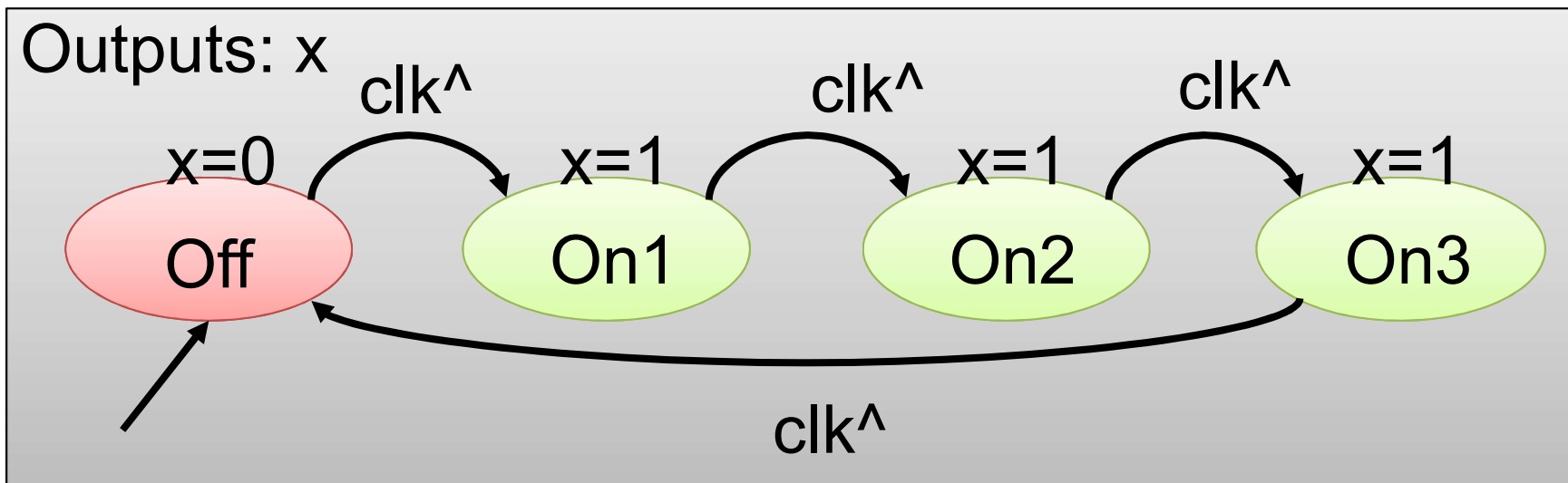
FSM Example: 0,1,1,1,repeat

- Step1: Want 0, 1, 1, 1, 0, 1, 1, 1, ...
 - Each value for one clock cycle
- Step2: Can describe as FSM: Four states,
Transition on rising clock edge to next state



FSM Example: 0,1,1,1,repeat

- Step1: Want 0, 1, 1, 1, 0, 1, 1, 1, ...
 - Each value for one clock cycle
- Step2: Can describe as FSM: Four states, Transition on rising clock edge to next state

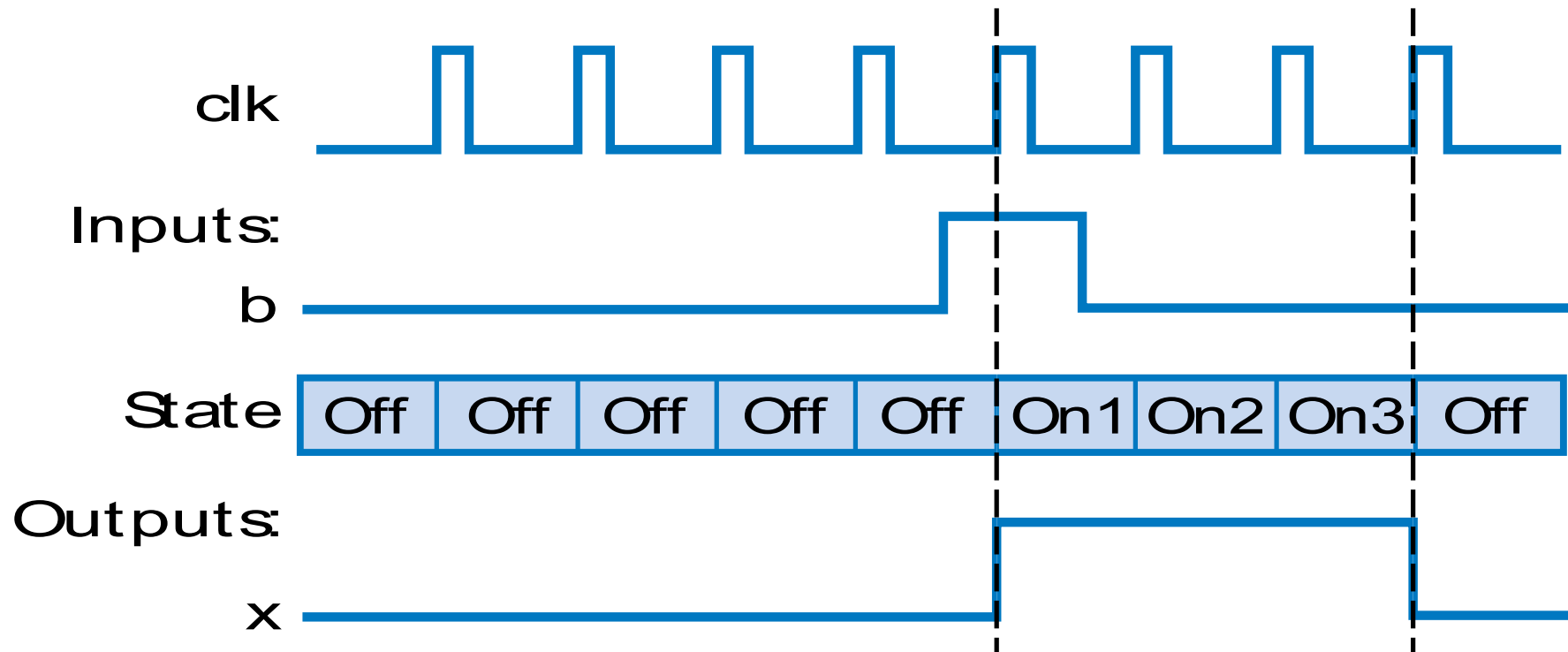


FSM Example 7
(General Example: Three-Cycles High
Laser Timer)

Extend FSM to Three-Cycles High Laser Timer

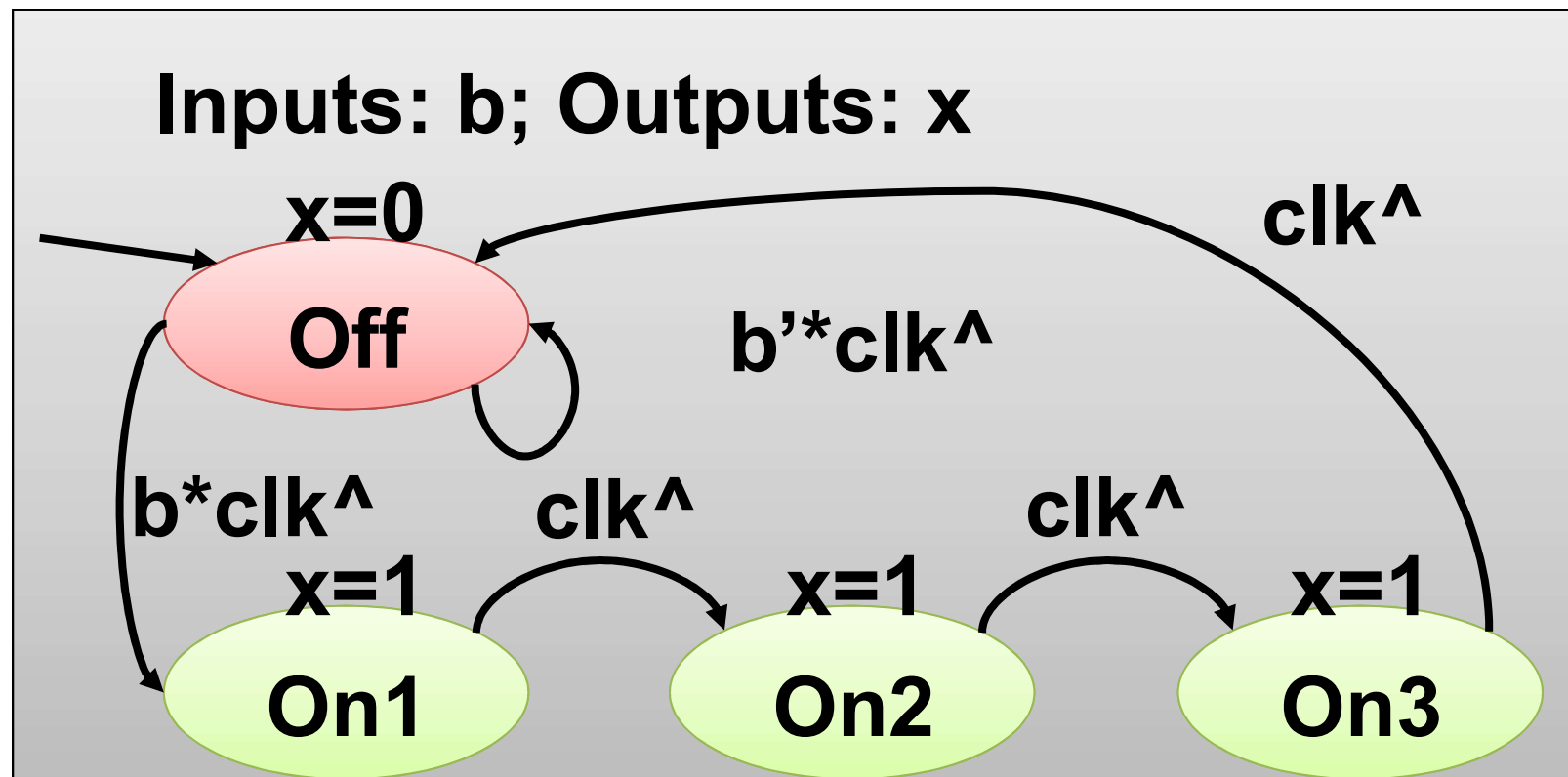
- Step1: Specify the problem in English like Stmts
 - Four states: Wait in “Off” state while b is 0 (b')
 - When $b=1$ (& rising clock edge), transition to On1
 - Sets $X=1$
 - On next two clock edges, transition to On2, then On3, which also set $x=1$
 - So $x=1$ for three cycles after button pressed

Extend FSM to Three-Cycles High Laser Timer



Specify: FSM of Three-Cycles High Laser Timer

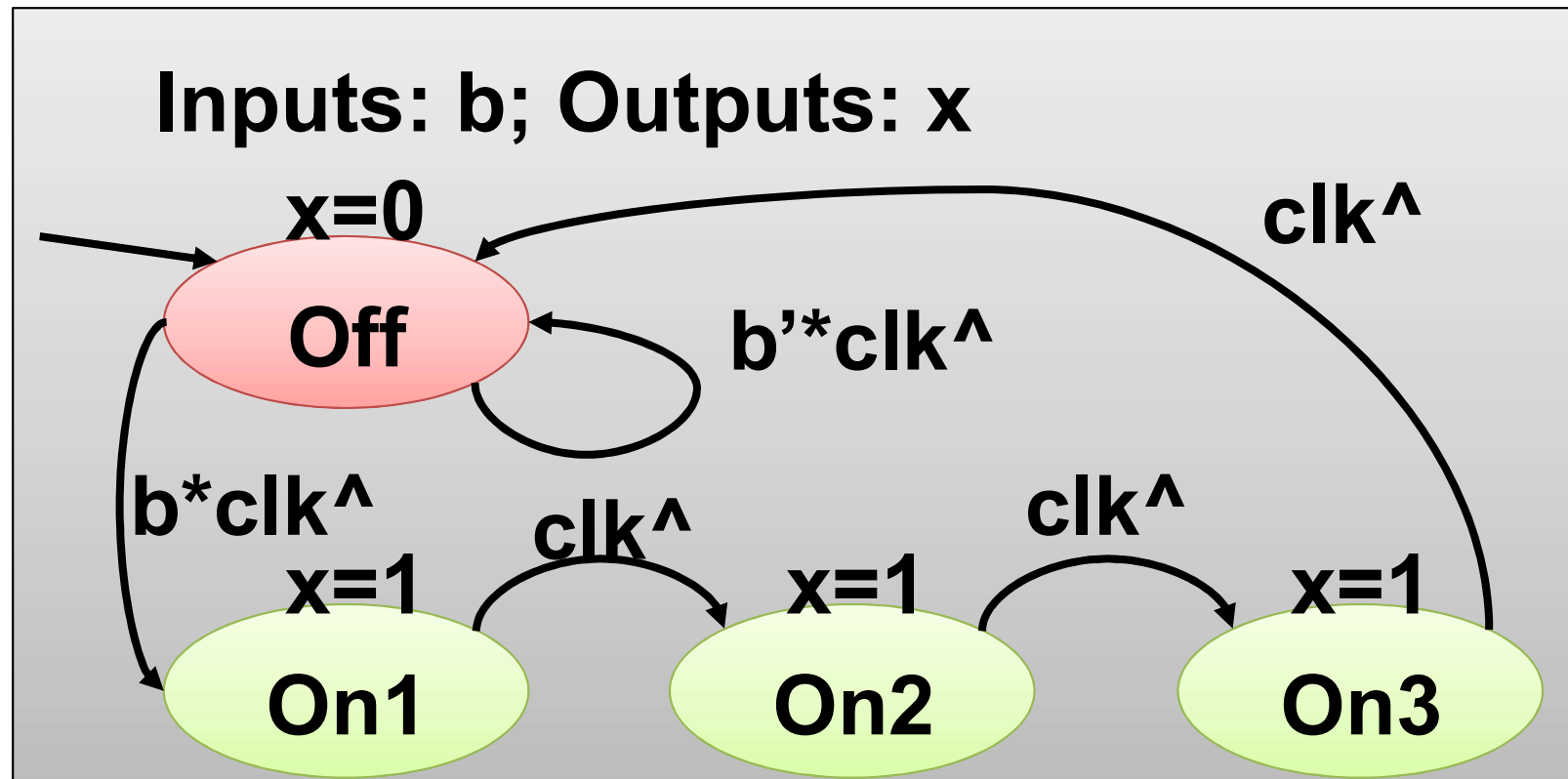
Step2: Model the problem using FSM



FSM Simplification: Rising Clock Edges Implicit

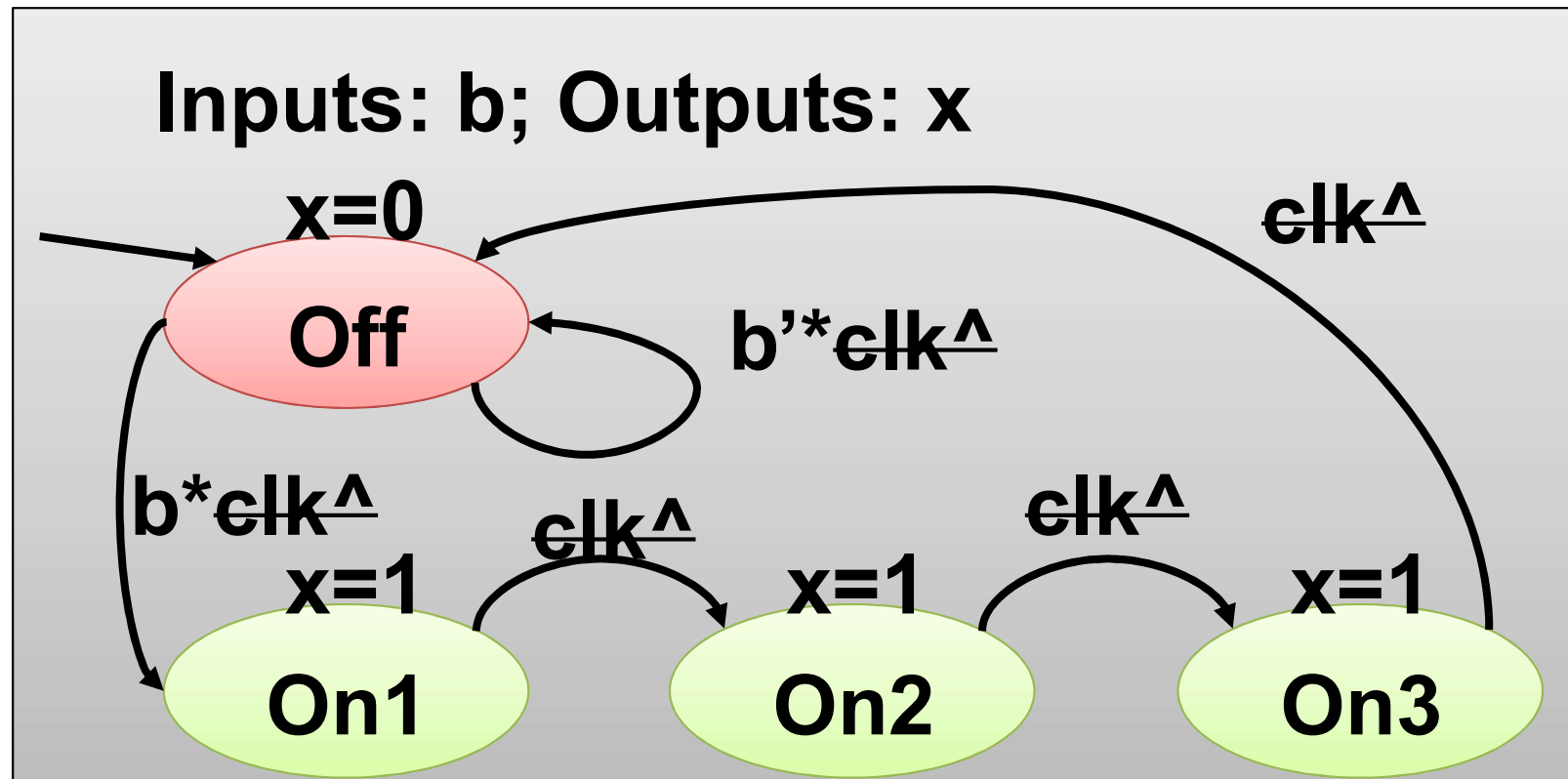
- Showing rising clock on every transition: **cluttered**
- Make implicit -- assume every edge has rising clock
- What if we wanted a transition *without* a rising edge
 - Asynchronous FSMs -- less common, and advanced topic
 - We consider *synchronous* FSMs
 - **All transition on rising edge**

Extend FSM to Three-Cycles High Laser Timer



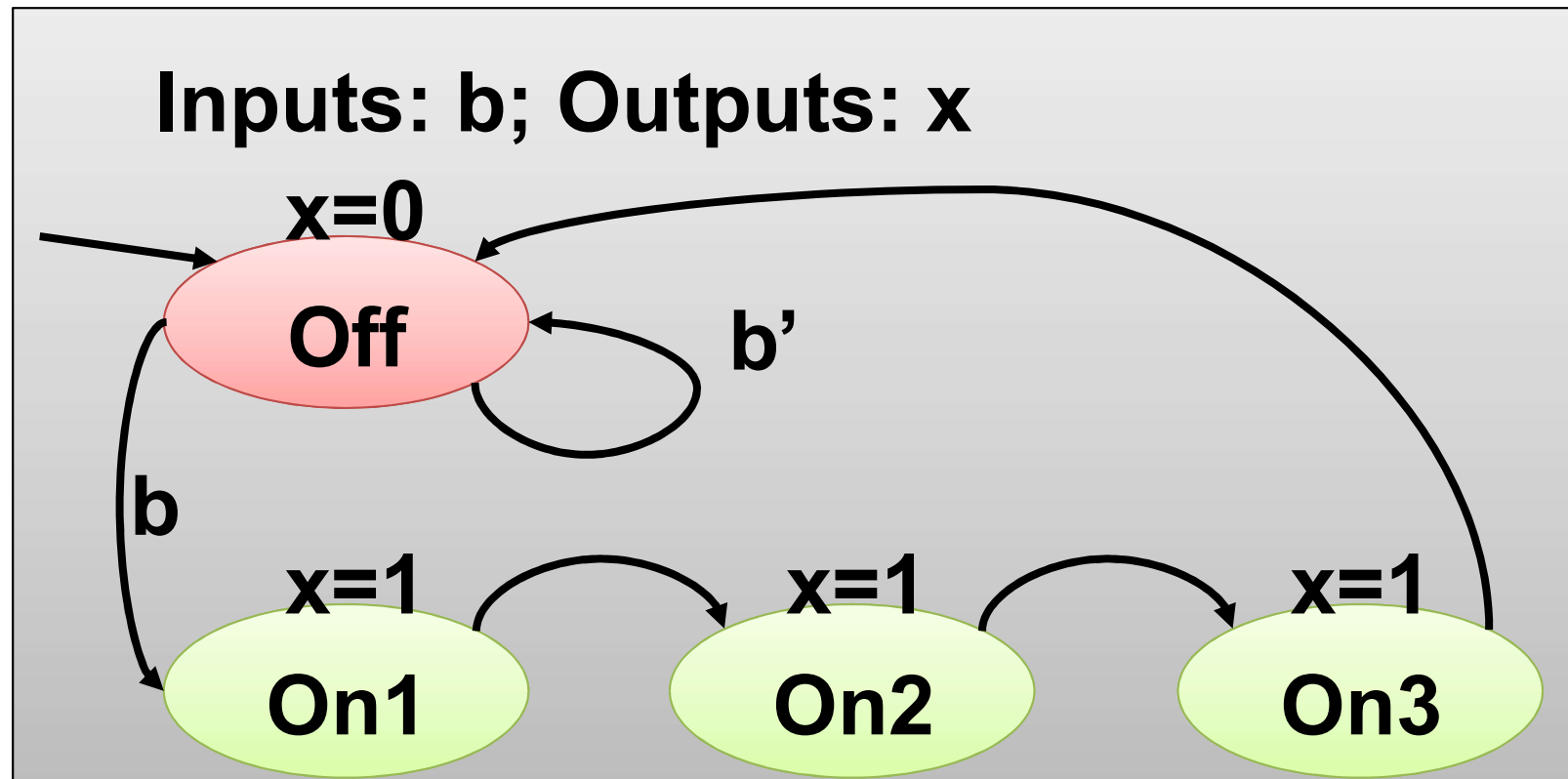
Note: Transition with no associated condition thus transitions to next state on next clock cycle

Extend FSM to Three-Cycles High Laser Timer



Note: Transition with no associated condition thus transitions to next state on next clock cycle

Extend FSM to Three-Cycles High Laser Timer



Note: Transition with no associated condition thus transitions to next state on next clock cycle

FSM Implementation

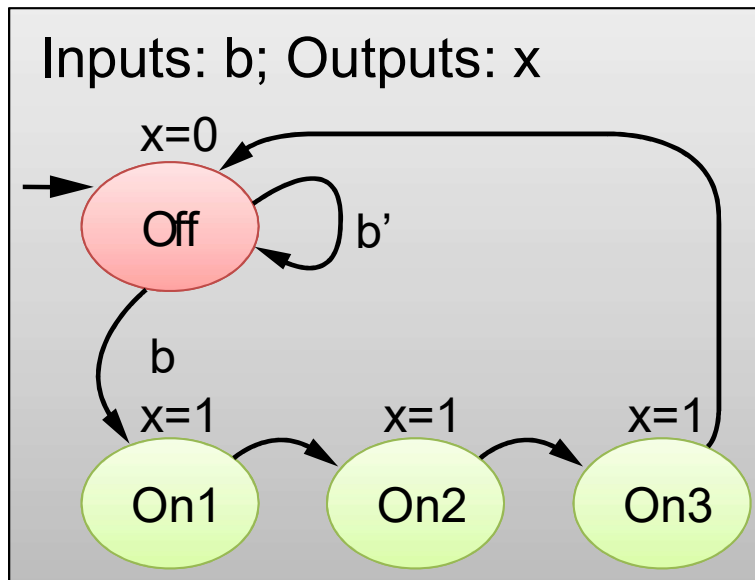
Step 4: Derive number of FF require
Four States: 2 FF

Step 5: State Encoding

Off: 00 On1: 01

On2: 10 on3: 11

Step 6: Derivation of Next State



PS	b	NS	X
00	0	00	0
00	1	01	0
01	x	10	1
10	x	11	1
11	x	00	1

FSM Implementation

PS	b	NS	X
00	0	00	0
00	1	01	0
01	x	10	1
10	x	11	1
11	x	00	1

Step 7: Derive for NS logic, OP logic

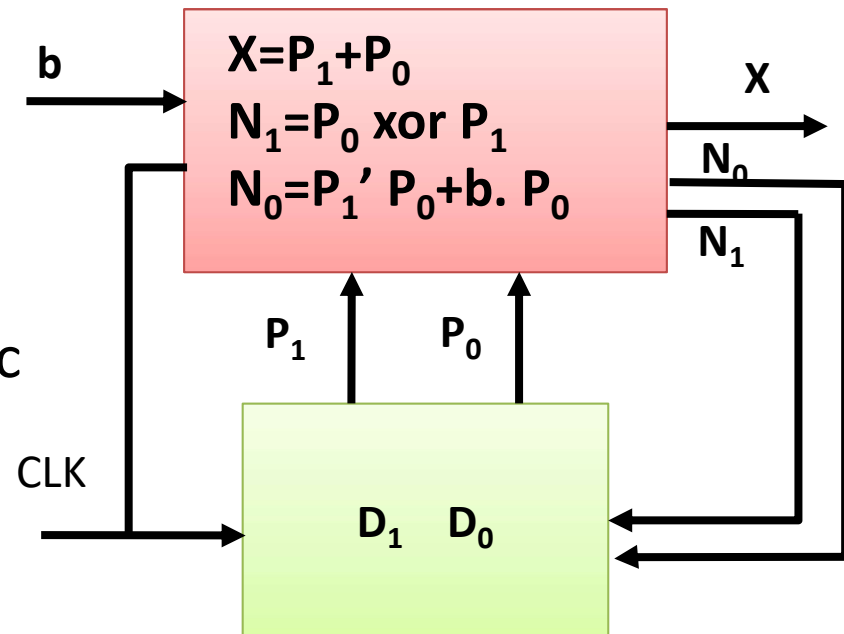
$$N_1 = P_0 \text{ xor } P_1$$

$$N_0 = P_1' P_0 + b \cdot P_0$$

$$X = P_1 + P_0$$

Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock



**Once we specify FSM for a
problem/system**

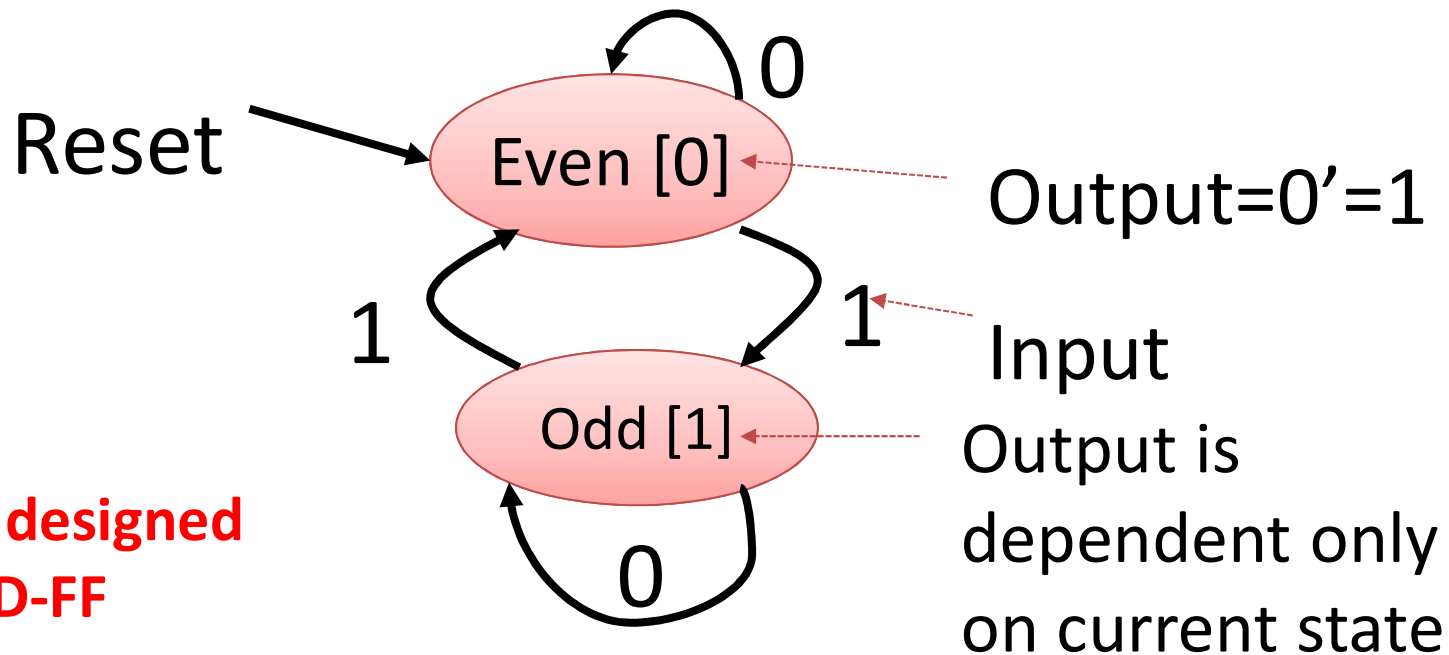
=== >

**Implementation is not
difficult**

FSM Example 8: Parity Encoder

FSM Example 8: Parity Encoder

- Input: 1 or 0 // entering as stream
- Output: output a 1 when total number of 1 is even

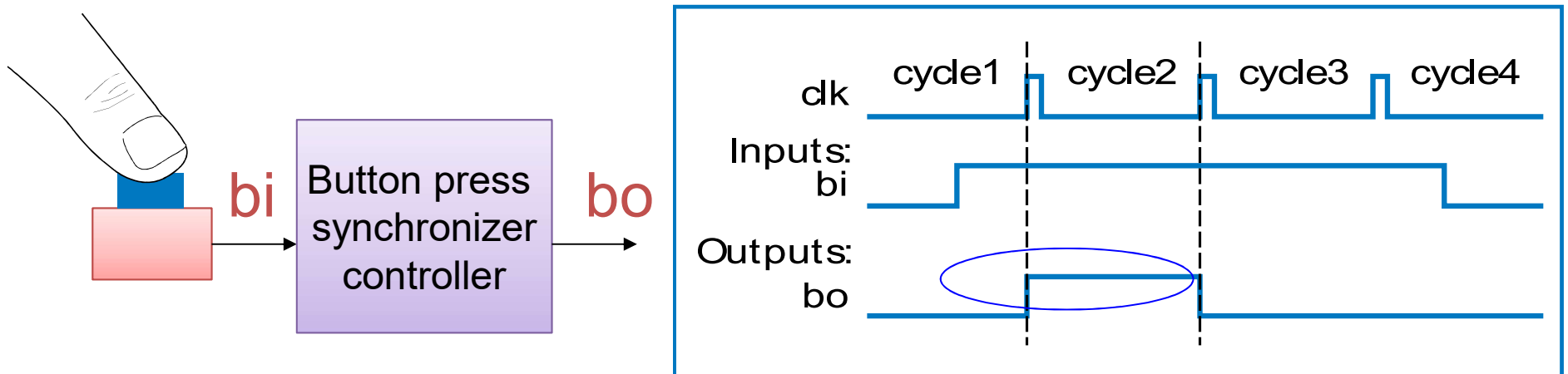


**T- FF : designed
using D-FF**

FSM Example 9: Button Press Synchronizer

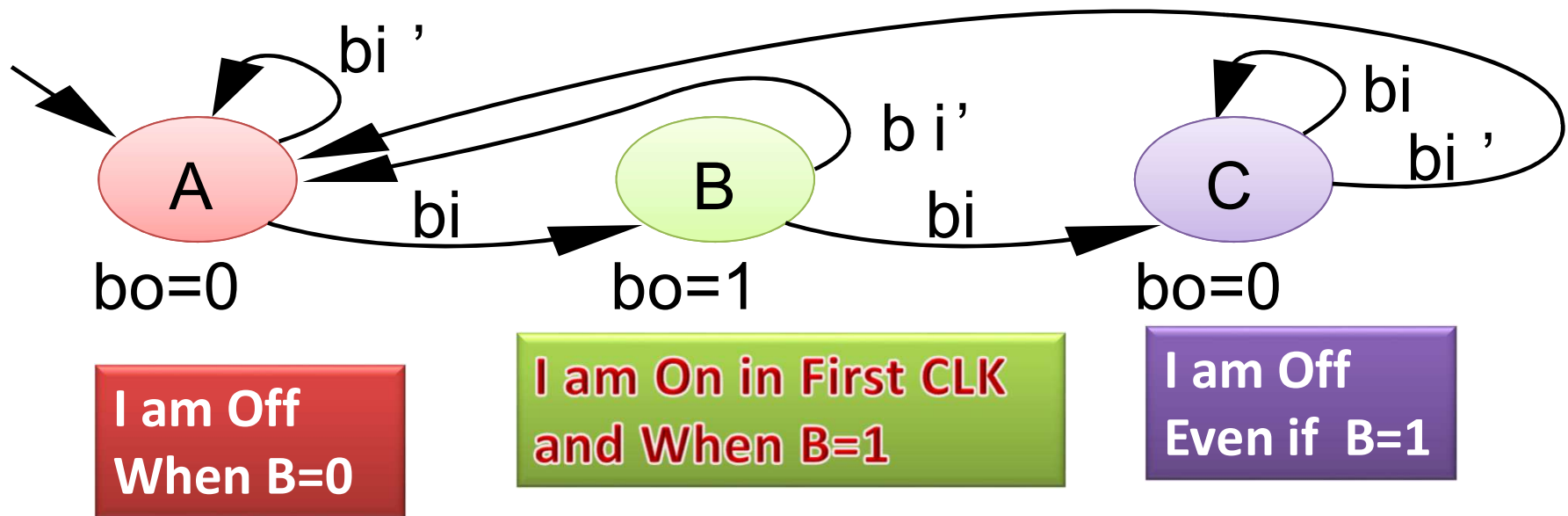
FSM : Button Press Synchronizer

- **English Language Specification**
- We want simple sequential circuit
 - Converts button press to single cycle duration
 - Regardless of length of time that button actually pressed



FSM : Button Press Synchronizer

FSM inputs: bi ; FSM outputs: bo

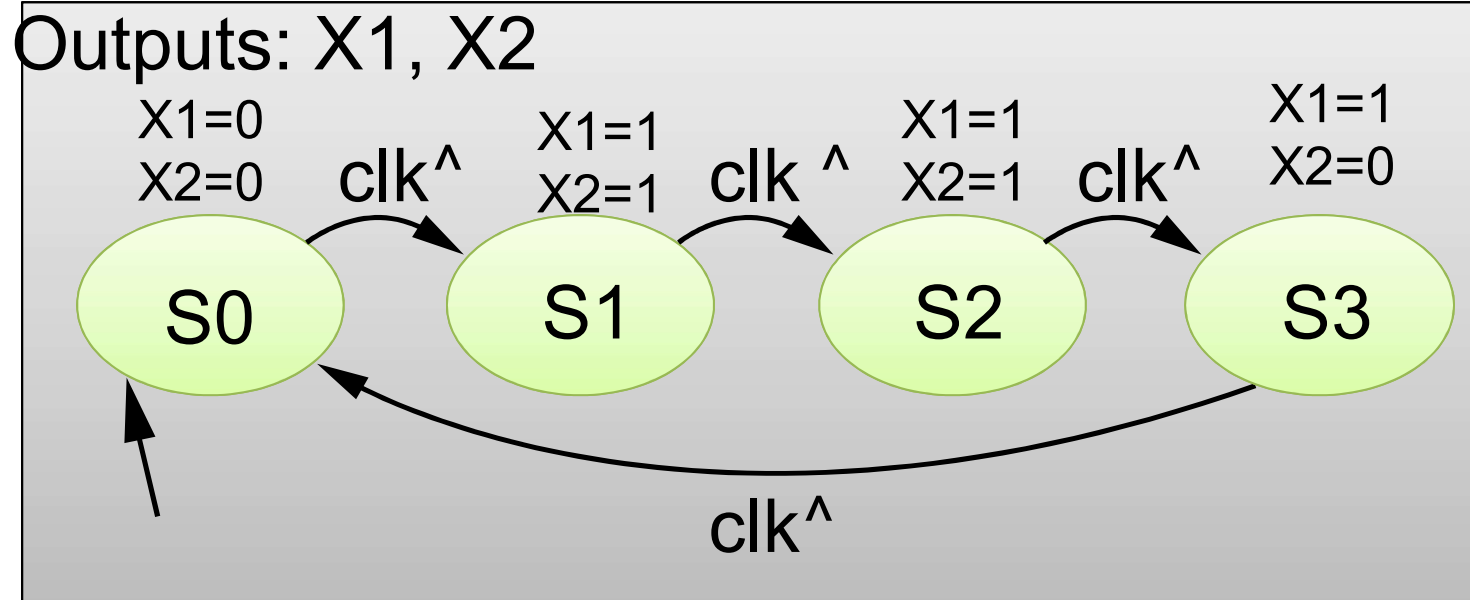


Step 1: Design FSM

FSM Example 10: Sequence generator

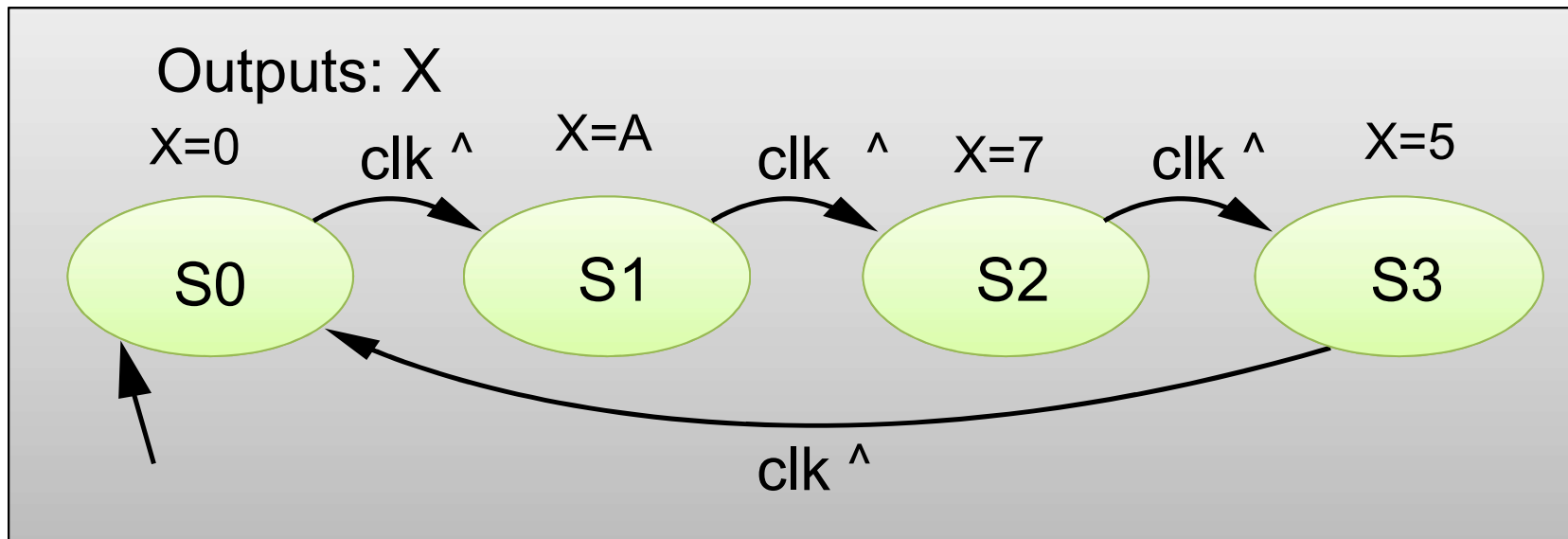
FSM Example 10: Sequence generator

- Generate two output sequence
 - $X1 = 0111\dots$ repeat
 - $X2 = 0110\dots$ repeat



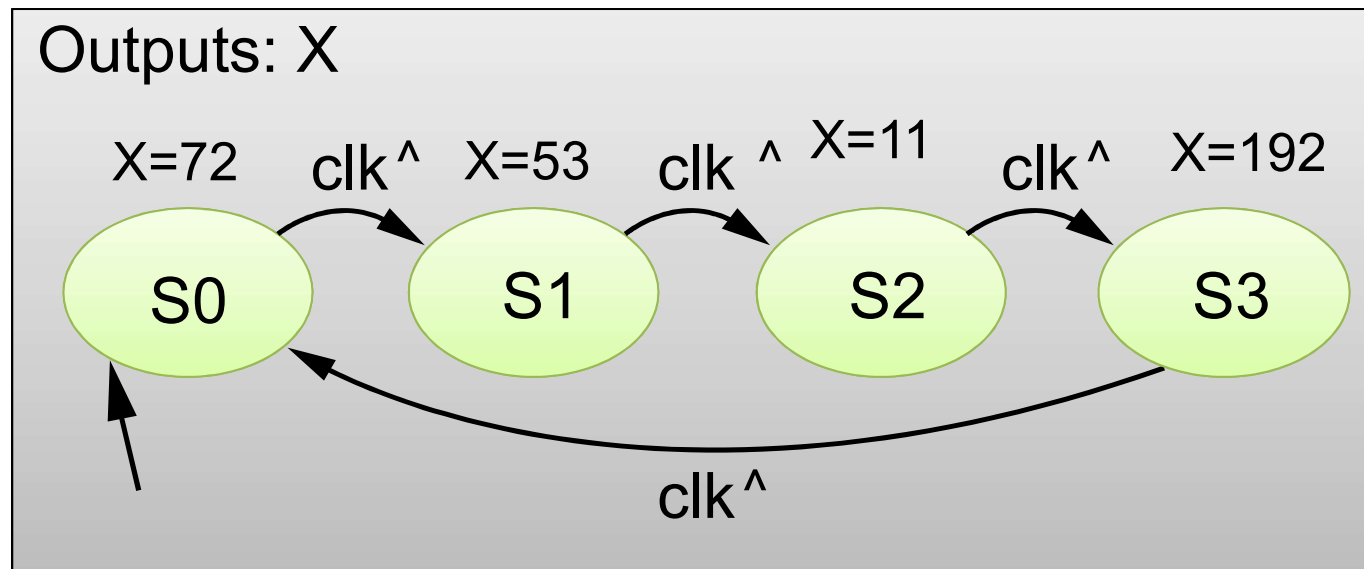
FSM Example 10-E1: Sequence generator

- Generate 4 bit integer output sequence
 - $X = 0, A, 7, 5 \dots$ repeat



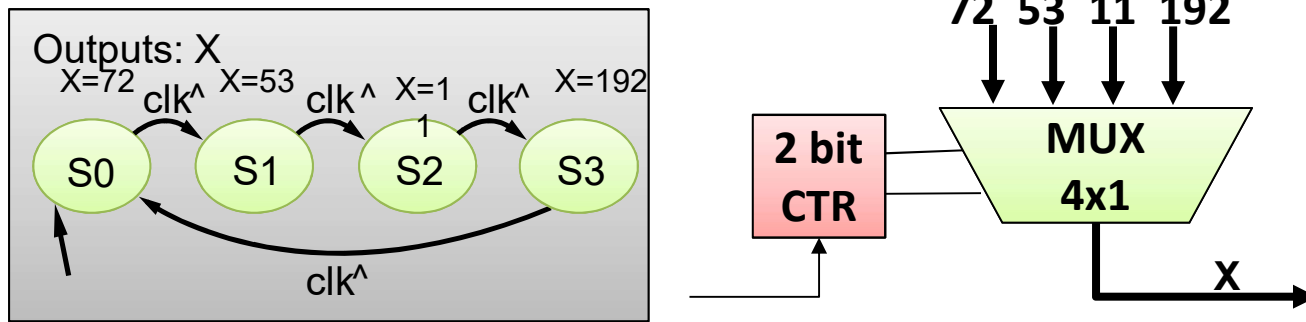
FSM Example 10-E2: Sequence generator

- Generate 8 bit integer output sequence
 - $X = 72, 53, 11, 192, \dots$ repeat



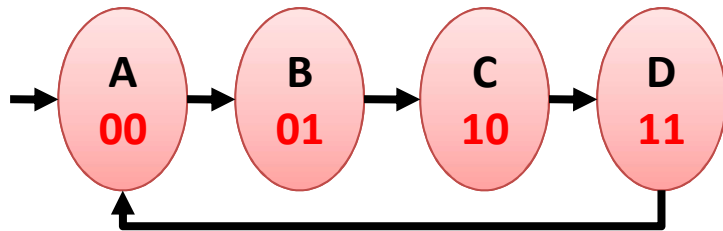
FSM Example 10-E2: Sequence generator

- Generate 8 bit integer output sequence
 - $X = 72, 53, 11, 192, \dots$ repeat



A,B,C,D values can be hardwired

FSM Example 10-E2: Sequence generator



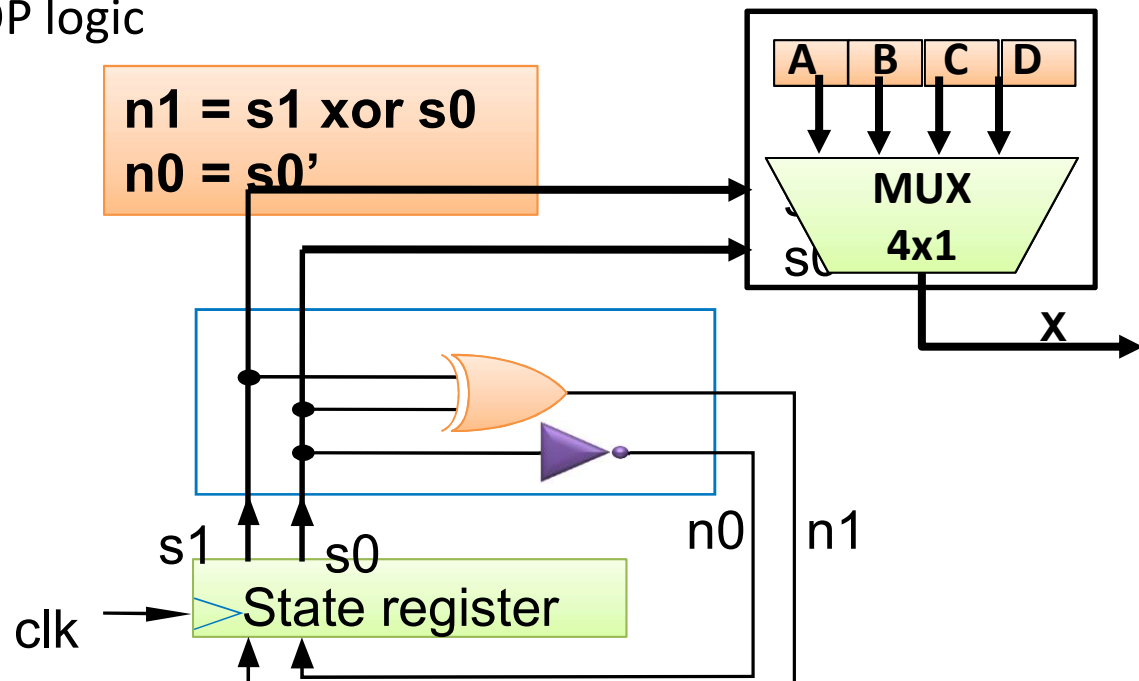
Step 8: Realize FSM

by Connecting the NS, OP logic, State Reg and Clock

Step 7: Derive for NS logic, OP logic

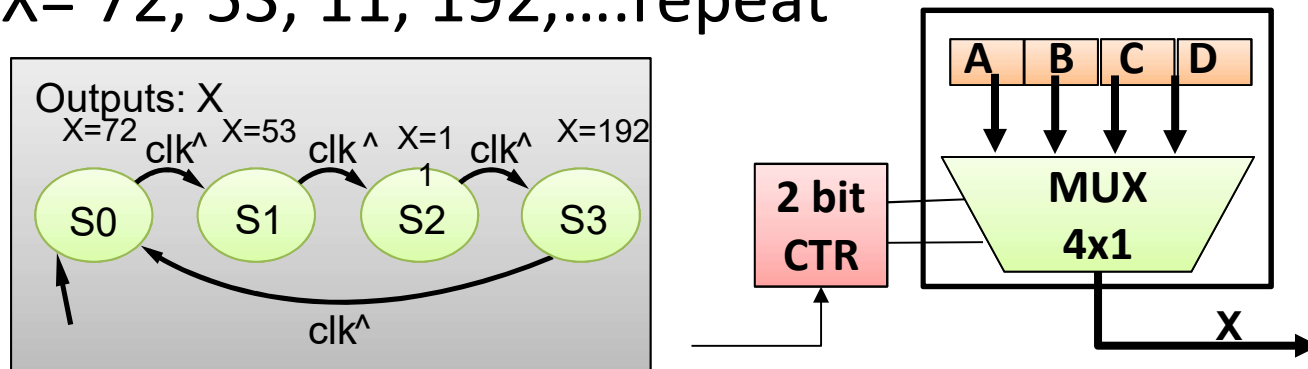
NS : $n1 = s1 \text{ xor } s0$; $n0 = s0'$

	PS		NS	
	s1	s0	n0	n1
A	0	0	0	1
B	0	1	1	0
C	1	0	1	1
D	1	1	0	0



FSM Example 10-E2: Sequence generator

- Generate 8 bit integer output sequence
 - $X = 72, 53, 11, 192, \dots$ repeat



A,B,C,D values can be stored in 4 registers
(With Mux it acts as memory)

FSM Output logic
may be simpler than
Register and Mux

Simpler to Implement
Output logic: Eight 2
input binary function

Thanks